UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
PRÓ-REITORIA DE GRADUAÇÃO
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA
CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

ROSENDO LUCAS SANTANA DA COSTA

UM GUIA PARA TESTES DE SOFTWARE EM APLICAÇÕES *FRONT-END*

PAU DOS FERROS
2024

ROSENDO LUCAS SANTANA DA COSTA

UM GUIA PARA TESTES DE SOFTWARE EM APLICAÇÕES *FRONT-END*

Monografia apresentada à Universidade Federal Rural do Semi-Árido como requisito parcial para obtenção do título de Bacharel em Engenharia de Software.

Orientador: Prof. Dr. Alysson Filgueira Milanez - UFERSA

PAU DOS FERROS

2024

© Todos os direitos estão reservados a Universidade Federal Rural do Semi-Árido. O conteúdo desta obra é de inteira responsabilidade do (a) autor (a), sendo o mesmo, passível de sanções administrativas ou penais, caso sejam infringidas as leis que regulamentam a Propriedade Intelectual, respectivamente, Patentes: Lei n° 9.279/1996 e Direitos Autorais: Lei n° 9.610/1998. O conteúdo desta obra tomar-se-á de domínio público após a data de defesa e homologação da sua respectiva ata. A mesma poderá servir de base literária para novas pesquisas, desde que a obra e seu (a) respectivo (a) autor (a) sejam devidamente citados e mencionados os seus créditos bibliográficos.

C837g Costa, Rosendo Lucas Santana da.
Um guia para testes de software em aplicações front-end web / Rosendo Lucas Santana da Costa. - 2024.
65 f. : il.

Orientador: Alysson Filgueira Milanez.
Monografia (graduação) - Universidade Federal Rural do Semi-árido, Curso de Engenharia de Software, 2024.

1. teste de software. 2. front-end. 3. Web. 4. guia. I. Milanez, Alysson Filgueira, orient. II. Título.

Ficha catalográfica elaborada por sistema gerador automáto em conformidade com AACR2 e os dados fornecidos pelo) autor(a).
Biblioteca Campus Pau dos Ferros
Bibliotecária: Erlanda Maria Lopes da Silva
CRB: 15/966

O serviço de Geração Automática de Ficha Catalográfica para Trabalhos de Conclusão de Curso (TCC´s) foi desenvolvido pelo Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo (USP) e gentilmente cedido para o Sistema de Bibliotecas da Universidade Federal Rural do Semi-Árido (SISBI-UFERSA), sendo customizado pela Superintendência de Tecnologia da Informação e Comunicação (SUTIC) sob orientação dos bibliotecários da instituição para ser adaptado às necessidades dos alunos dos Cursos de Graduação e Programas de Pós-Graduação da Universidade.

ROSENDO LUCAS SANTANA DA COSTA

UM GUIA PARA TESTES DE SOFTWARE EM APLICAÇÕES *FRONT-END*

Monografia apresentada à Universidade Federal Rural do Semi-Árido como requisito parcial para obtenção do título de Bacharel em Engenharia de Software.

Defendida em: 24 / 10 / 2024

BANCA EXAMINADORA

Prof. Dr. Alysson Filgueira Milanez (Orientador)
UFERSA

Prof. Dr. David Candeia Medeiros Maia
IFPB

Prof. Me. Kláudio Henrique Mascarenhas Medeiros
UNIFACISA

AGRADECIMENTOS

Neste espaço, gostaria de expressar minha profunda gratidão a todas as pessoas que contribuíram para a realização deste trabalho.

Primeiramente, gostaria de expressar minha profunda gratidão à minha mãe, Keila Karina, e à minha tia, Kaliani Cristina. O apoio incondicional que ambas me ofereceram foram essenciais ao longo de toda a minha jornada acadêmica. A força e a dedicação que demonstraram foram fundamentais para que eu pudesse superar os desafios e alcançar meus objetivos. Sou eternamente grato por tudo o que fizeram por mim e por sempre acreditarem em meu potencial.

Aos meus amigos e colegas, gostaria de expressar minha profunda gratidão pela companhia e pelo incentivo constantes durante toda a minha jornada. Os momentos de descontração e alegria que compartilhamos foram fundamentais para aliviar a pressão dos desafios enfrentados ao longo desse percurso.

Agradeço também ao meu orientador, Prof. Dr. Alysson Filgueira Milanez, por sua orientação, paciência e expertise ao longo do desenvolvimento deste trabalho. Suas valiosas sugestões e críticas construtivas foram essenciais para que eu pudesse aprimorar minhas ideias e alcançar um resultado satisfatório.

Por fim, agradeço à banca avaliadora, por dedicarem seu tempo para analisar meu trabalho e por suas contribuições enriquecedoras durante a defesa. Agradeço a todos pela atenção e pelos comentários que certamente me ajudarão a crescer como profissional.

Muito obrigado!

EPÍGRAFE

“O futuro é muito brilhante para ser abandonado.”

(My Hero Academia)

RESUMO

O teste de software é um processo fundamental para garantir a qualidade de um produto, atuando como uma ferramenta essencial na prevenção de erros e na melhoria contínua. Portanto, é importante que testes sejam aplicados de maneira abrangente em produtos de software, incluindo todas as camadas e componentes, para assegurar um desenvolvimento robusto. No desenvolvimento de aplicações Web, o *front-end* desempenha um papel central, por ser a camada que interage diretamente com o usuário final. Essa interface não apenas exibe informações, mas também deve responder de maneira eficiente às ações do usuário, proporcionando uma experiência agradável, intuitiva e sem falhas. Portanto, garantir a qualidade do *front-end* é crítico, uma vez que problemas nessa camada podem comprometer a usabilidade e, consequentemente, a satisfação do usuário. Além disso, erros não detectados no *front-end* podem impactar diretamente a reputação do produto e da organização responsável. Dado o impacto significativo do *front-end* no desenvolvimento de aplicações Web, o objetivo deste estudo é criar um material de apoio voltado para desenvolvedores, com o propósito de facilitar o processo de aprendizagem na área de testes de software, com ênfase no *front-end* Web. A análise de outros estudos que têm o mesmo objetivo de promover boas práticas de teste de software, servindo como guia ou material de apoio para facilitar a aplicação de testes, revelou a importância de criar um material que funcione como um projeto aberto. Esse projeto seria disponibilizado para que desenvolvedores não apenas pudessem consumi-lo, mas também contribuíssem para a sua evolução. Dessa forma, o conteúdo se manteria atualizado e fundamentado na experiência atual dos desenvolvedores.

Palavras-chave: teste de software; *front-end*; Web; guia.

ABSTRACT

Software testing is a fundamental process for ensuring the quality of a product, acting as an essential tool for error prevention and continuous improvement. Therefore, it is important that testing is applied comprehensively to software products, including all layers and components, to ensure robust development. In the development of web applications, the front-end plays a central role, as it is the layer that interacts directly with the end user. This interface not only displays information, but also must respond efficiently to user actions, providing a pleasant, intuitive and flawless experience. Therefore, ensuring the quality of the front-end is critical, since problems in this layer can compromise usability and, consequently, user satisfaction. Furthermore, undetected errors in the front-end can directly impact the reputation of the product and the organization responsible. Given the significant impact of the front-end on the development of Web applications, the objective of this study is to create support material aimed at developers, with the purpose of facilitating the learning process in the area of software testing, with an emphasis on the Web front-end. The analysis of other studies that have the same objective of promoting good software testing practices, serving as a guide or support material to facilitate the application of tests, revealed the importance of creating a material that functions as an open project. This project would be made available so that developers could not only consume it, but also contribute to its evolution. In this way, the content would remain up-to-date and based on the current experience of developers.

Keywords: software testing; front-end; Web; guide.

LISTA DE FIGURAS

Figura 1 – Formulário de pesquisa - Área com maior experiência 29
Figura 2 – Formulário de pesquisa - Perfil do participante 30
Figura 3 – Formulário de pesquisa - Adoção de testes de software 30
Figura 4 – Formulário de pesquisa - Dificuldades das equipes com teste de software) . 31
Figura 5 – Formulário de pesquisa - Dificuldades das equipes sem teste de software . . 31
Figura 6 – Formulário de pesquisa - Habilidade em teste de software no *front-end* . . . 32
Figura 7 – Formulário de pesquisa - Aplicação de teste manuais (equipes sem teste de software) 33
Figura 8 – Formulário de Pesquisa - Aplicação de testes manuais (equipes com teste de software) 33
Figura 9 – Formulário de Pesquisa - Ferramentas utilizadas pelos participantes 34
Figura 10 – Sumário do guia 35
Figura 11 – Segunda seção do guia 36
Figura 12 – Terceira seção do guia 36
Figura 13 – Quarta seção do guia 37
Figura 14 – Quinta seção do guia 38
Figura 15 – Tela do sistema MeContrata 39
Figura 16 – Guia: Dependências de pacotes para o Jest 41
Figura 17 – Guia: Arquivo de definições do Jest 41
Figura 18 – Guia: Exemplo de componente a ser testado 43
Figura 19 – Guia: Dependências necessárias para o Jest no *front-end* 44
Figura 20 – Guia: Exemplo de teste construídos com o Jest 45
Figura 21 – Perfil dos desenvolvedores com relação à prática de teste 46
Figura 22 – Perfil dos desenvolvedores sobre o conteúdo e organização do guia 47
Figura 23 – Interesse dos participantes em contribuir com o guia 47
Figura 24 – Interesse dos participantes em compartilhar o guia 48

LISTA DE QUADROS

Quadro 1 – Comparação de trabalhos 25

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
E2E	end-to-end
GUI	Graphical User Interface
QA	Quality Assurance
SQA	Software Quality Assurance
TCE-MT	Tribunal de Contas de Mato Grosso
TI	Tecnologia da Informação
UFERSA	Universidade Federal Rural do Semi-Árido

SUMÁRIO

1 INTRODUÇÃO 13

1.1 Problemática 13

1.2 Justificativa 14

1.3 Objetivos 15

1.3.1 Objetivo geral 15

1.3.2 Objetivos específicos 15

1.4 Contribuição da presente pesquisa 16

1.5 Organização do trabalho 16

2 FUNDAMENTAÇÃO TEÓRICA 17

2.1 Desenvolvimento *front-end* Web 17

2.2 Qualidade de software 18

2.3 Garantia de qualidade de software 18

2.4 Teste de software 19

2.5 Testes de aplicações Web *front-end* 20

3 TRABALHOS RELACIONADOS 21

3.1 Importância de testes unitários no desenvolvimento 21

3.2 Automação de testes em aplicações Web utilizando uma abordagem *ad-hoc* 22

3.3 Criação de um guia de teste de software a partir da análise de repositórios do Github 22

3.4 A importância dos testes automatizados 23

3.5 Proposição de um guia de automação de testes de software para o TCE-MT: um estudo de caso 23

3.6 *Selenium framework for Web automation testing: A systematic literature review* 24

3.7 Comparação com os trabalhos relacionados 25

4 METODOLOGIA 26

4.1 Pesquisar trabalhos correlacionados 26
4.2 Elaborar formulário de pesquisa 27
4.3 Aplicar e avaliar formulário de pesquisa 27
4.4 Desenvolver guia de apoio ao desenvolvimento de testes 28
5 RESULTADOS E DISCUSSÕES 29
5.1 Análise do cenário atual no desenvolvimento de testes 29
5.2 Guia para testes Front-End 34
5.3 Aplicação do guia para testes *front-end* 41
5.4 Pesquisa de *feedback* 45
6 CONCLUSÕES E TRABALHOS FUTUROS 49
REFERÊNCIAS 50
APÊNDICE A PESQUISA SOBRE O TESTE DE SOFTWARE NO *FRONT-END* 52
APÊNDICE B FEEDBACK PARA O GUIA DE DESENVOLVIMENTO DE TESTES *FRONT-END* WEB 62

1 INTRODUÇÃO

O desenvolvimento de sistemas web impulsiona uma ampla variedade de aplicações, atendendo desde plataformas de e-commerce e redes sociais até sistemas de gestão empresarial e ferramentas colaborativas de trabalho remoto. Com isso, um dos pontos fortes que fará uma aplicação se destacar é a experiência do usuário (Ijaz; Andlib, 2014). Assim, ao focarmos no *front-end* desses sistemas, que atua na interface entre o usuário e as funcionalidades da aplicação, identificamos uma área de grande impacto na experiência do usuário. Sendo necessário ter um nível de qualidade em seu funcionamento e design (Pacheco; Garbatov; Goulão, 2021).

Uma maneira eficaz de alcançar um nível ideal de qualidade é por meio da aplicação de testes no sistema, com foco especial no *front-end*, embora não se restrinja exclusivamente a ele nesta análise. No desenvolvimento da Interface Gráfica do Usuário (do inglês Graphical User Interface (GUI)), os testes desempenham um papel crucial, frequentemente exigindo execução manual devido aos aspectos abstratos envolvidos. Esses testes garantem que a GUI seja intuitiva, responsiva e minimize a ocorrência de falhas, essencial para uma experiência de usuário satisfatória e para a conformidade com os requisitos de design e usabilidade (Nass; Alégroth; Feldt, 2021).

Os testes manuais, embora ainda comuns nesse segmento, apresentam limitações, como ser um processo penoso e custoso financeiramente (Alberto *et al.*, 2020). Para contornar esses problemas, é possível aplicar testes automatizados em outras camadas do sistema, especialmente na camada de serviços, responsável pela coleta e exibição de dados. Porém, mesmo com a existência dos testes automatizados, muito tempo e recursos são demandados, que muitas vezes são vistos negativamente (Borjesson; Feldt, 2012), mesmo que tais práticas tragam benefícios ao longo prazo.

1.1 Problemática

Os testes de software desempenham um papel fundamental na garantia da eficiência e eficácia dos sistemas (Maldonado *et al.*, 2004). A garantia de qualidade de um produto de software está diretamente ligada à sua capacidade de alcançar um produto minimamente funcional (Bueno; Campelo, 2011), sem falhas que possam comprometer sua utilidade e confiabilidade. Em um cenário em que até mesmo pequenos erros podem resultar em grandes prejuízos, especialmente dependendo da criticidade do software, a presença de falhas pode ter impactos significativos para

os usuários finais e para a reputação da empresa desenvolvedora.

Dado isso, é imprescindível que os testes sejam aplicados ao longo de todo o processo de desenvolvimento, garantindo que o software seja testado adequadamente (Júnior, 2011). Investir em testes desde as fases iniciais do desenvolvimento não apenas ajuda a identificar e corrigir problemas precocemente, mas também contribui para reduzir custos e riscos associados a falhas detectadas tardiamente (Júnior, 2011).

No desenvolvimento de aplicações *front-end* web, a implementação de testes de software pode, à primeira vista, parecer mais complexa e desafiadora. Isso ocorre devido à necessidade de lidar com interfaces dinâmicas, simular interações de usuários, gerenciar estados e garantir compatibilidade entre diferentes navegadores e dispositivos. Essa aparente complexidade pode desmotivar alguns desenvolvedores a adotar a prática, apesar de sua importância para assegurar a qualidade do produto final. No entanto, é crucial reconhecer os benefícios significativos que os testes podem trazer para a qualidade e a confiabilidade do software (Campopiano, 2022).

Desenvolver aplicações *front-end* envolve lidar com interações complexas entre o código HTML, CSS e JavaScript, além de integrar com Application Programming Interface (API) e serviços externos. A complexidade aumenta à medida que as aplicações crescem em funcionalidades e tamanho, tornando essencial a implementação de testes para garantir que todas as partes do sistema funcionem como esperado (Campopiano, 2022).

1.2 Justificativa

Para entender melhor esse cenário, é essencial analisar como a prática de testes de software está disseminada no mercado de trabalho na produção de sistemas. Esta análise proporciona *insights* valiosos para a construção de um guia prático que auxilie na aplicação eficaz dos testes, fortalecendo assim a qualidade e a eficácia das práticas de desenvolvimento *front-end*.

Os testes de software desempenham um papel fundamental na garantia da estabilidade e confiabilidade das aplicações *front-end* (Campopiano, 2022). Ao investigar como esses testes estão sendo adotados e implementados na indústria, é possível identificar padrões de sucesso, desafios comuns enfrentados pelos desenvolvedores e áreas onde melhorias são necessárias.

Ao construir um guia prático baseado nesses *insights*, os desenvolvedores podem ser capacitados com boas práticas para planejar, implementar e manter testes de software eficientemente. Isso não apenas ajuda a reduzir a incidência de bugs e falhas no código *front-end*, mas

também otimiza o tempo de desenvolvimento ao longo do ciclo de vida do projeto.

Além da importância de disponibilizar materiais de apoio ao aprendizado de testes de software, a criação de um projeto aberto em português representa um diferencial significativo em relação a outras iniciativas com o mesmo objetivo. Um exemplo é o livro de Kinsbruner e Bahmutov (2022), que orienta sobre como maximizar a eficácia de *frameworks* de automação de testes em aplicações web, auxiliando desenvolvedores na integração da automação em seus fluxos de trabalho e na melhoria da confiabilidade e do *feedback* nas alterações de código.

Um projeto aberto que incentiva a participação da comunidade brasileira é essencial para a evolução do desenvolvimento de software, atuando como um centro de conhecimento e experiências compartilhadas entre os profissionais da área. Ao promover a colaboração, esse projeto não apenas facilita a troca de ideias e melhores práticas, mas também fortalece a rede de contatos entre desenvolvedores, fomentando um ambiente de aprendizado contínuo.

1.3 Objetivos

Nesta seção, são descritos os objetivos geral e específicos do trabalho.

1.3.1 Objetivo geral

Este projeto visa desenvolver um guia para testes no *front-end* Web, que servirá como uma ferramenta de aprendizado para desenvolvedores que estão ingressando no campo do teste de software.

O principal objetivo do guia é auxiliar desenvolvedores iniciantes, simplificando a introdução de testes no fluxo de trabalho. O foco está em desmistificar o processo de teste, apresentando exemplos práticos e explicações claras para facilitar a adoção dessas práticas. Ao incorporar uma cultura de testes desde o início do planejamento de um produto de software, espera-se que os desenvolvedores aumentem a confiabilidade do sistema, reduzir a ocorrência de falhas e melhorar a experiência do usuário final.

1.3.2 Objetivos específicos

Para alcançar o objetivo descrito acima foram definidos os seguintes objetivos específicos:

- Elaborar um formulário de pesquisa com questões qualitativas e quantitativas para avaliar o cenário atual do desenvolvimento de testes de software;

- Aplicar formulário com profissionais da área e avaliar dados coletados;
- Definir o sistema que será utilizado como ferramenta para os exemplos práticos de testes;
- Definir tecnologias adotadas pelo guia para construção dos testes automatizados;
- Elaborar guia de testes.

1.4 Contribuição da presente pesquisa

Este trabalho apresenta um guia destinado a auxiliar desenvolvedores na implementação de testes de software. O guia fornecerá detalhes sobre a aplicação de testes no contexto do desenvolvimento Web, com foco específico em aplicações *front-end*, visando fortalecer a construção de produtos com maior garantia de qualidade.

Além de apoiar profissionais em suas práticas diárias, este guia também se propõe a ser um recurso inicial para estudos na área de testes de software pelos desenvolvedores, proporcionando conhecimento fundamental e incentivando o aprofundamento no assunto, demonstrando os valores positivos da aplicação de testes.

1.5 Organização do trabalho

O presente trabalho encontra-se estruturado da seguinte forma: no Capítulo 2 são apresentados os referencias teóricos necessários para o embasamento do trabalho; o Capítulo 3 apresenta os principais trabalhos relacionados à presente pesquisa; já o Capítulo 4 descreve a metodologia aplicada no projeto; no Capítulo 5, são apresentados e discutidos os resultados obtidos no trabalho; por fim, no Capítulo 6, são apresentadas as considerações finais e os trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, serão abordados tópicos fundamentais para a compreensão deste trabalho. Serão explorados aspectos essenciais que contribuem significativamente para o entendimento do conteúdo, com foco em quatro seções principais: desenvolvimento *front-end* Web (seção 2.1), que detalha as tecnologias que sustentam a criação de aplicações para a Web; garantia de qualidade de software (seção 2.3), que abrange as práticas e critérios para garantir um software eficiente e confiável; testes de software (seção 2.4), onde é apresentando esse segmento no desenvolvimento de software e sua importância; e, por fim, testes de aplicações Web (seção 2.5), com foco específico nas técnicas e ferramentas empregadas para verificar a qualidade e o desempenho de aplicações desenvolvidas para a Web.

2.1 Desenvolvimento *front-end* Web

Com o aumento do acesso à internet e dos dispositivos móveis, o desenvolvimento de sistemas web cresceu significativamente. Esses sistemas podem ser acessados diretamente por meio de um navegador, eliminando a necessidade de instalação nas máquinas locais, o que facilita as implantações e direciona os esforços para a gestão do servidor (Sommerville, 2011).

Os sistemas Web podem ser definidos em dois segmentos: *back-end* e *front-end*. O *back-end* é responsável por realizar o processamento bruto de dados, como a manipulação de informações em um banco de dados (Liu *et al.*, 2017). Enquanto o *front-end* é a parte que irá fazer a fronteira entre o usuário e o sistema, fornecendo uma interface gráfica, que normalmente é processada pela máquina do usuário (Liu *et al.*, 2017). Este é um fator importante para as aplicações modernas, pois terá um papel fundamental para experiência do usuário (Lorençon; Gibertoni, 2021).

É importante ressaltar que a qualidade do *front-end* tem papel importante na experiência do usuário no sistema, pois uma aplicação bem construída, com design intuitivo e agradável, fará uma grande diferença na retenção de usuários. Com isso, tal segmento deve passar por etapas de validação e verificação, com intuito de contribuir com uma experiência eficiente e eficaz (Ijaz; Andlib, 2014).

Por fim, o desenvolvimento *front-end* Web é uma área que apresenta uma grande importância para conquistar os usuários, sendo de total interesse das empresas. Com isso, é crucial o investimento em ferramentas e profissionais capacitados para garantir a qualidade e eficiência

dos sistemas desenvolvidos (Ijaz; Andlib, 2014).

2.2 Qualidade de software

A qualidade de software é um aspecto fundamental para o sucesso de produtos digitais e está amplamente discutida tanto em pesquisas quanto em práticas de desenvolvimento. De acordo com Pressman e Maxim (2016), a qualidade de software refere-se ao grau em que o software atende aos requisitos funcionais e não funcionais estabelecidos, como confiabilidade, eficiência e usabilidade.

Muitas vezes, espera-se que a qualidade do software esteja associada exclusivamente à correta execução das funcionalidades do sistema, ou seja, à realização correta de uma operação específica. No entanto, a qualidade vai além da execução correta das funcionalidades e abrange também os requisitos não funcionais, que são igualmente importantes para a experiência geral do usuário e para o sucesso do software no mercado. Como afirma Sommerville (2011), a qualidade de software não se limita à implementação precisa das funcionalidades, mas envolve também atributos não funcionais que garantem o desempenho, a segurança e a robustez do sistema.

Dessa forma, uma funcionalidade não deve apenas realizar sua tarefa corretamente; ela também precisa fazê-lo de forma otimizada e confiável. Por exemplo, é essencial que o software garanta alta disponibilidade, além de assegurar a integridade e a confiabilidade dos dados processados. Esses atributos não funcionais são críticos para que o software atenda às expectativas dos usuários e mantenha-se competitivo no mercado, onde a experiência do usuário e a robustez do sistema desempenham papéis centrais na percepção de qualidade.

2.3 Garantia de qualidade de software

Ter um software de alta qualidade é fundamental para o sucesso de seu produto (Moser *et al.*, 2021). Para isso, tem-se a Garantia de Qualidade de Software (do inglês Software Quality Assurance (SQA)), um conjunto de atividades que seguem um padrão/modelo adotado para seu produto de software, visando alcançar requisitos mínimos para garantia de sua confiabilidade (Wong; Yu; Too, 2018).

É importante entender que a SQA não é responsável simplesmente por aplicar testes de software. Ela detém de atividades que levam a analisar todo o processo de software focando em sua melhoria (Wong; Yu; Too, 2018), pois tem o foco em prevenir as falhas com a evolução e o

amadurecimento do processo.

A SQA não se concentra exclusivamente na etapa de desenvolvimento do software com a aplicação de testes. A qualidade de software deve considerar todas as fases do processo em suas análises. Em qualquer fase do processo de desenvolvimento de software, podem surgir oportunidades de melhoria que proporcionem uma maior garantia de qualidade. Contudo, os testes ainda são extremamente importantes, especialmente os testes automatizados, uma vez que a automatização dos processos é crucial para prevenir falhas (Wong; Yu; Too, 2018).

Além das atividades comentadas, a equipe de SQA também é responsável por definições de padrões e diretrizes para o desenvolvimento de software (Wong; Yu; Too, 2018), como a análise de requisitos, a validação e verificação de funcionalidades e o monitoramento de métricas de qualidade.

2.4 Teste de software

A engenharia de software surgiu com a demanda por produtos de maior qualidade, em que as aplicações construídas apresentavam bastante falhas, com custos altos e entregues com atraso (Pressman; Maxim, 2016). Ela utiliza de princípios e técnicas das engenharias que iriam auxiliar a diminuir esses deméritos no desenvolvimento de software (Pressman; Maxim, 2016). Logo, foram definidos diversos processos a serem aplicados durante o ciclo de vida de um software, sendo um deles o teste de software (Maldonado *et al.*, 2004).

O objetivo principal é garantir que o sistema seja bem projetado e desenvolvido, de modo a atender às expectativas dos usuários e evitar falhas constantes durante seu ciclo de vida (Sommerville, 2011). A realização dos testes é de grande importância para o produto de software, pois suas atividades estão presentes ao longo de todo o ciclo de vida, com intuito de minimizar os erros e seus impactos em um sistema (Maldonado *et al.*, 2004).

O teste de software é um processo que irá avaliar um software, com objetivo de identificar possíveis erros e falhas (Maldonado *et al.*, 2004), algo que possa afetar a qualidade do produto final. Com uma variedade de técnicas e abordagens, como testes unitários e de integração, que são essenciais no desenvolvimento. Ademais, não se deve limitar apenas aos testes funcionais; também existem os testes de sistema, que têm a função de validar se a aplicação atende aos requisitos especificados (Maldonado *et al.*, 2004).

Nesta área, existem formas de aplicar automação dos testes por meio de ferramentas, algo que irá facilitar a construção e execução dos testes em determinadas circunstâncias, como

aplicar testes de regressão, em que não será necessário todo um processo manual de validação das funcionalidades e têm-se os casos de teste reaproveitados (Maldonado *et al.*, 2004). Entretanto, existem validações que não possuem muitas possibilidades de serem aplicadas de forma automática, pois os testes automatizados estão mais presentes na validação de código de baixo nível (Borjesson; Feldt, 2012).

Indo para o contexto do desenvolvimento de aplicações Web, no segmento do *front-end*, passa-se pela situação de muitos testes manuais serem executados. Isto ocorre em virtude de se trabalhar com o desenvolvimento de interfaces gráficas, em que é necessário avaliar a experiência dos usuários com a mesma, sendo algo mais abstrato, não tendo casos de teste claros, o que pode contribuir em uma maior demanda de tempo ao aplicar os testes no *front-end* e o torna uma tarefa maçante (Azzam *et al.*, 2022).

2.5 Testes de aplicações Web *front-end*

O desenvolvimento de aplicações Web tem avançado rapidamente, e com isso, a aplicação de testes de software tornou-se necessária para garantir a qualidade e confiabilidade (Moser *et al.*, 2021). No entanto, em aplicações Web, geralmente, não é realizado o investimento necessário em testes automatizados, com a maioria dos testes realizados manualmente (Moura, 2021).

Dado isso, ao longo da evolução de seu produto, é importante manter a qualidade do produto garantindo seu funcionamento e integridade, pois uma falha pode acarretar prejuízos financeiros e de imagem para a empresa (Campopiano, 2022). Com isso, a aplicação de testes automatizados é uma prática que pode ser adotada para garantir a qualidade do produto, pois permite a execução de testes de forma rápida e eficiente, além de permitir a execução de testes de regressão, fundamentais para garantir que as alterações realizadas no código não impactem negativamente no funcionamento do sistema (Moura, 2021).

Por fim, a prática de testes em aplicações Web é necessária tanto quanto em qualquer tipo de aplicação. Logo, é importante que se invista em testes para garantir a qualidade de seus produtos e a satisfação de seus clientes (Campopiano, 2022).

3 TRABALHOS RELACIONADOS

A elaboração de materiais de estudo para auxiliar no desenvolvimento de testes de software no contexto de aplicações Web é um tema que tem sido abordado por diversos autores como Campopiano (2022) e Moura (2021). Neste capítulo, serão apresentados alguns trabalhos relacionados que abordam a importância dos testes de software, as práticas adotadas no desenvolvimento *front-end* Web e as ferramentas utilizadas para a realização dos testes.

3.1 Importância de testes unitários no desenvolvimento

No trabalho elaborado por Campopiano (2022), é destacada a importância dos testes unitários, com um enfoque especial no esclarecimento dessa prática essencial no desenvolvimento de software. O autor apresenta exemplos práticos que demonstram como aplicar testes unitários em funções escritas em JavaScript, utilizando a ferramenta Jest. Mediante uma abordagem clara e didática, o estudo ilustra como os testes podem ser facilmente integrados ao fluxo de desenvolvimento, proporcionando uma compreensão mais profunda dos benefícios dessa prática.

Semelhante ao presente trabalho, o trabalho elaborador por Campopiano (2022) apresenta exemplos de como escrever testes unitários para funções específicas, o trabalho discute a importância de testar cada componente isoladamente para garantir que o comportamento esperado seja mantido ao longo do desenvolvimento. O uso de Jest, uma das bibliotecas mais populares para testes em JavaScript, é detalhado, mostrando suas funcionalidades, como *mocks* e asserções, que tornam o processo de teste mais eficiente. Com isso, o autor não só promove a adoção de testes unitários, mas também oferece um guia prático que pode ser seguido por desenvolvedores em diferentes níveis de experiência, reforçando a relevância da qualidade do código no desenvolvimento de aplicações robustas e confiáveis.

Assim como o presente trabalho, o estudo elaborado por Campopiano (2022) também serve como uma ferramenta de aprendizado, proporcionando o conhecimento necessário para a introdução aos testes de software. Além disso, o trabalho de Campopiano (2022) oferece uma base sólida para iniciantes que desejam aprofundar seus conhecimentos na área, contribuindo para o aprimoramento contínuo das habilidades em testes de software.

3.2 Automação de testes em aplicações Web utilizando uma abordagem *ad-hoc*

Com o material elaborado por Moura (2021), pode-se compreender a importância dos testes automatizados no desenvolvimento de aplicações Web. O trabalho foca em apresentar uma ferramenta que facilita a aplicação da abordagem *Ad-hoc* em testes automatizados, destacando como essa metodologia pode ser integrada ao processo de desenvolvimento. Além disso, o autor demonstra o funcionamento de testes escritos com a ferramenta Cypress, uma solução robusta e popular para testes *end-to-end*, e aborda como a ferramenta elaborada pelo autor se encaixa nesse processo.

O estudo não apenas descreve a funcionalidade do Cypress, mas também ilustra, por meio de exemplos práticos, como os testes automatizados podem ser implementados eficazmente. Ao correlacionar a aplicação da ferramenta com os conceitos discutidos, o trabalho oferece uma visão clara de como os testes automatizados podem melhorar a qualidade do software. Dessa forma, o material contribui para a promoção de boas práticas no desenvolvimento de aplicações Web, incentivando a adoção de testes automatizados como parte essencial do ciclo de vida do software.

3.3 Criação de um guia de teste de software a partir da análise de repositórios do Github

O trabalho de Sampaio (2022) concentra-se em explorar o potencial dos dados extraídos de repositórios de software, com um enfoque especial nos repositórios hospedados no GitHub, como fonte de apoio para a tomada de decisões em projetos de desenvolvimento. Essa pesquisa investiga como informações derivadas desses repositórios podem oferecer *insights* valiosos para desenvolvedores. Ao analisar esses dados, o autor busca evidenciar padrões que não apenas auxiliem na escolha de tecnologias e metodologias, mas também promovam práticas de desenvolvimento mais eficientes e alinhadas com as necessidades do ciclo de vida do software.

Sampaio (2022) realizou uma análise detalhada de repositórios públicos, buscando identificar práticas, processos e *frameworks* que contribuem para a aplicação eficaz de testes de software. A pesquisa resultou em um guia voltado a programadores iniciantes, com o objetivo de ajudá-los a adotar boas práticas e escolher *frameworks* eficientes para criar testes que aprimorem a qualidade de seus projetos.

Semelhante ao presente trabalho, o estudo realizado por Sampaio (2022) também foca na produção de um guia para contribuir com a formação de programadores iniciantes. No entanto,

ao contrário do trabalho de Sampaio (2022), o projeto atual é um recurso aberto, o que permite que o guia não fique limitado a uma versão específica definida durante o estudo. Essa abordagem possibilita que a comunidade de desenvolvedores contribua e atualize continuamente o conteúdo, mantendo-o sempre relevante e alinhado com as práticas mais recentes.

3.4 A importância dos testes automatizados

No trabalho desenvolvido por Bernardo e Kon (2008), são destacados aspectos fundamentais sobre a automação de testes no desenvolvimento de software, evidenciando tanto os benefícios dessa prática quanto as limitações dos testes manuais. O autor argumenta que, embora os testes manuais possam ser rápidos e intuitivos para pequenas verificações, eles tendem a ser altamente ineficientes e suscetíveis a erros quando se trata de grandes sistemas ou de processos repetitivos, resultando em atrasos no cronograma de desenvolvimento e na entrega de produtos com qualidade duvidosa.

Para mitigar esses problemas, Bernardo e Kon (2008) apresenta a automação de testes como uma solução eficaz, descrevendo-a como uma prática que permite executar grandes baterias de testes de forma rápida e consistente, com menor esforço humano. O trabalho oferece exemplos práticos de automação utilizando ferramentas como JUnit, um *framework* para testes unitários em Java, e Selenium, uma ferramenta popular para testes de interface gráfica de aplicações web. Essas ferramentas são destacadas por sua capacidade de integrar-se facilmente ao fluxo de desenvolvimento e por possibilitar a criação de testes mais complexos, como simulações de usuários interagindo com sistemas web ou testes de carga com grande volume de dados.

Além disso, o autor explora como a automação de testes se alinha com as metodologias ágeis, que promovem ciclos rápidos de desenvolvimento e entregas contínuas. Ao adotar testes automatizados, as equipes de desenvolvimento podem garantir que as novas funcionalidades sejam constantemente verificadas e que o sistema mantenha sua integridade mesmo com mudanças frequentes.

3.5 Proposição de um guia de automação de testes de software para o TCE-MT: um estudo de caso

O trabalho desenvolvido por Donato (2023) planeja propor um guia de automação de testes para sistemas do Tribunal de Contas de Mato Grosso (TCE-MT), oferecendo uma

abordagem sistemática e prática para a implementação de testes automatizados em sistemas complexos. O estudo baseia-se em uma análise detalhada dos benefícios da automação em comparação com o processo manual.

Durante a pesquisa, foram realizados comparativos entre os métodos de verificação manual e automatizada em funções específicas dos sistemas. Esses comparativos se concentraram em três fatores principais: o tempo de execução dos testes, o esforço humano necessário para realizar cada tipo de verificação e a facilidade de repetição do processo, especialmente em cenários de regressão, onde a mesma função precisa ser testada repetidamente após alterações no código.

Em síntese, o guia proposto visa fornecer uma referência prática e eficiente para que o TCE-MT possa integrar a automação de testes de software em seus processos, visando otimizar a qualidade, a segurança e a eficiência dos sistemas de tecnologia da informação da instituição.

O trabalho de Donato (2023), assim como o presente estudo, visa apresentar um guia para testes de software, no entanto, ele é especificamente focado no TCE-MT. Em contraste, este o presente trabalho oferece um guia mais abrangente, que não está vinculado a um sistema específico, visando atender a todos os desenvolvedores.

3.6 *Selenium framework for Web automation testing: A systematic literature review*

Thooriqoh, Annisa e Yuhana (2021) realizam uma revisão sistemática da literatura sobre o *framework* Selenium, com foco em seu uso para automação de testes em aplicações web. O estudo fornece um resumo abrangente das pesquisas existentes, destacando como o Selenium, um *framework* de código aberto amplamente utilizado em conjunto com diversas linguagens de programação, pode ser aplicado na automação de casos de teste.

O trabalho seguiu um rigoroso processo metodológico de revisão sistemática, em que 16 estudos publicados entre 2009 e 2020 foram selecionados para análise. Os resultados indicam que o Selenium é amplamente utilizado para automação web, conseguindo testar não apenas a funcionalidade completa dos aplicativos. Mas também pode ser mesclado com métodos e algoritmos, como inteligência artificial (Thooriqoh; Annisa; Yuhana, 2021).

Thooriqoh, Annisa e Yuhana (2021) também sugerem que, em pesquisas futuras, a aplicação do Selenium em automação de testes deve focar em aprimorar a efetividade e a manutenção do *framework*, explorando seu uso em diferentes metodologias e propondo melhorias que otimizem ainda mais sua adequação para automação de testes web.

3.7 Comparação com os trabalhos relacionados

Foram encontrados alguns trabalhos que discutem maneiras de aplicar testes automatizados. No entanto, como pode ser observado no Quadro 1, a maioria das pesquisas não é focada especificamente em aplicações Web. Além disso, algumas dessas publicações abordam apenas sistemas ou metodologias específicas, limitando a abrangência dos resultados. Essa lacuna na literatura ressalta a necessidade de mais estudos direcionados ao desenvolvimento de testes automatizados para aplicações Web, uma vez que essa área apresenta particularidades que podem exigir abordagens e técnicas específicas para garantir a eficácia e a eficiência dos testes.

Quadro 1 – Comparação de trabalhos

Trabalhos	Exemplos práticos	Fornecimento de material como projeto aberto	Material para sistemas específicos	Material focado em aplicações Web
Campopiano (2022)	X	-	-	-
Moura (2021)	X	-	X	X
Bernardo e Kon (2008)	X	-	-	-
Donato (2023)	X	-	X	-
Thooriqoh, Annisa e Yuhana (2021)	-	-	-	-
Sampaio (2022)	X	-	-	-
Presente trabalho	X	X	-	X

Fonte: Autoria própria (2024)

Com esse objetivo em mente, o projeto foi concebido como uma iniciativa de projeto aberto, disponível no GitHub[1], permitindo que outros contribuam com melhorias, atualizações e novas seções. Essa abordagem colaborativa visa não apenas enriquecer o conteúdo do guia, mas também fomentar a troca de experiências e boas práticas entre desenvolvedores, ampliando o alcance e a aplicabilidade dos testes automatizados em diferentes contextos de desenvolvimento.

[1] Disponível em: https://rosluc.github.io/front_test_guide/

4 METODOLOGIA

A metodologia da pesquisa foi estruturada em dois momentos principais. Primeiramente, foi realizado um estudo sobre trabalhos relacionados ao contexto de testes de software no desenvolvimento *front-end* Web, envolvendo a consulta a artigos, monografias e livros. Este estudo visou aprofundar o entendimento atual das práticas adotadas, identificando tanto os sucessos quanto as lacunas existentes.

A análise dos dados obtidos permitiu investigar as razões pelas quais algumas empresas e equipes ainda não incorporam práticas de teste efetivas nesse contexto. Essa investigação visou encontrar as motivações por trás das decisões de não adotar e identificar potenciais dificuldades em implantar os testes de software.

Em um segundo momento, com base nas análises realizadas, foi desenvolvido um material de estudo. Este material tem como propósito auxiliar na implantação e na compreensão dos testes no desenvolvimento Web, contribuindo para a disseminação das boas práticas e das vantagens associadas ao teste de software.

Para isto, uma série de passos foram necessários:

- Elaborar formulário de pesquisa;
- Aplicar pesquisa com profissionais da área e avaliar dados coletados;
- Desenvolver guia de apoio ao desenvolvimento de testes.

4.1 Pesquisar trabalhos correlacionados

Foi realizada uma pesquisa exploratória em diversas fontes, incluindo artigos científicos, livros, blogs especializados e repositórios de código, visando identificar trabalhos relacionados ao tema de testes de aplicações Web *front-end*. A literatura revisada abrangeu diferentes abordagens adotadas no processo de teste, destacando a importância da automação de testes e da integração contínua no ciclo de desenvolvimento de software.

Além disso, foram analisados estudos que abordam o uso de ferramentas populares, como Jest e Cypress, e o impacto dessas práticas na eficiência e qualidade das entregas. A pesquisa buscou tanto fundamentos teóricos quanto exemplos práticos, a fim de construir um panorama abrangente sobre o campo dos testes *front-end*.

4.2 Elaborar formulário de pesquisa

Por meio da ferramenta *Google Forms*, foi elaborado um formulário de pesquisa com perguntas abertas e fechadas (Apêndice A), com objetivo de levantar informações sobre a realidade das práticas de teste de software no desenvolvimento *front-end* Web, bem como identificar os principais desafios enfrentados pelas equipes de desenvolvimento.

Assim, conseguimos levantar informações sobre a adoção de testes automatizados, a utilização de ferramentas específicas, e a percepção dos profissionais sobre a importância dos testes de software. Essas informações foram fundamentais nas tomadas de decisão durante a elaboração do guia. O formulário foi construído com quatro seções principais, contendo questões quantitativas e qualitativas, para mapear o cenário atual e as dificuldades enfrentadas pelos desenvolvedores em relação aos testes de software.

A primeira seção visa traçar o perfil do participante, identificando sua área de atuação, as tecnologias que utiliza e seu nível de conhecimento na área de testes. A segunda seção foca em entender a visão do participante sobre o cenário atual dos testes de software, coletando informações sobre as ferramentas utilizadas em projetos, práticas adotadas, dificuldades enfrentadas e o domínio das técnicas de teste. A terceira seção visa obter dados sobre a prática de testes no contexto do *front-end*, identificando a frequência de sua aplicação e os problemas que impedem sua implementação. Por fim, a quarta e última seção é destinada a comentários e sugestões adicionais dos participantes.

4.3 Aplicar e avaliar formulário de pesquisa

O formulário de pesquisa (Apêndice A) foi aplicado aos profissionais da área de desenvolvimento de software, com foco em desenvolvimento *front-end* Web; mas, também, com profissionais de outras áreas, como desenvolvimento *back-end* e testes de software. O formulário foi amplamente divulgado entre os alunos da Universidade Federal Rural do Semi-Árido (UFERSA), profissionais que atuam no desenvolvimento de software e por meio de redes sociais. Essa abordagem visa alcançar uma variedade de respondentes, enriquecendo a diversidade dos dados coletados. A coleta de informações foi realizada de forma anônima, assegurando a privacidade dos participantes e permitindo que compartilhassem suas opiniões e experiências de maneira livre e aberta.

Com o formulário recebendo um total de 32 respostas, foi realizada uma análise dos

resultados obtidos, com o objetivo de identificar padrões, tendências e possíveis lacunas nas respostas dos participantes. Essa análise permitiu identificar as principais dificuldades enfrentadas pelas equipes de desenvolvimento em relação à adoção de práticas de teste de software, além de compreender as motivações que levam à decisão de não as adotar.

4.4 Desenvolver guia de apoio ao desenvolvimento de testes

Com base nos resultados obtidos na pesquisa e na análise dos trabalhos relacionados, foi elaborado um guia para apoiar o desenvolvimento de testes de software no contexto de aplicações Web *front-end*. O guia abrange as principais práticas e técnicas de teste, destacando ferramentas populares e boas práticas para a integração de testes no ciclo de desenvolvimento. Além disso, apresenta métodos essenciais, como testes unitários, testes de integração e testes *end-to-end*.

O principal objetivo do guia é auxiliar desenvolvedores iniciantes, simplificando a introdução de testes no fluxo de trabalho. O foco está em desmistificar o processo de teste, apresentando exemplos práticos e explicações claras para facilitar a adoção dessas práticas. Ao incorporar uma cultura de testes desde o início, espera-se que os desenvolvedores aumentem a confiabilidade do software, reduzir a ocorrência de falhas e melhorar a experiência do usuário final.

5 RESULTADOS E DISCUSSÕES

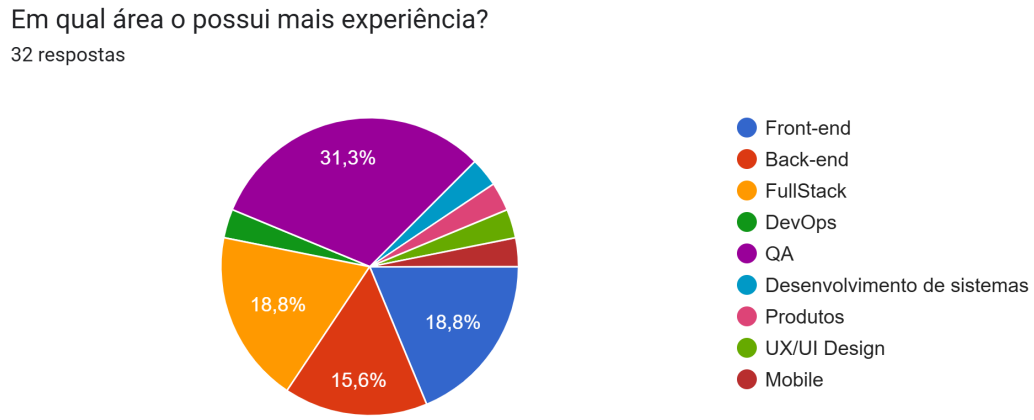
Neste capítulo, são apresentados os resultados obtidos a partir da elaboração do guia para testes *front-end* Web, cujo objetivo é fornecer um material de apoio para desenvolvedores que desejam iniciar seus estudos em testes automatizados no *front-end*.

5.1 Análise do cenário atual no desenvolvimento de testes

Visando construir um material de apoio voltado à aplicação de testes no desenvolvimento de aplicações web, tornou-se essencial realizar uma pesquisa com profissionais da área. Essa pesquisa permitiu analisar o cenário atual, identificando padrões, desafios e barreiras enfrentados no desenvolvimento de testes de software em aplicações Web.

Dado isso, foi realizada uma análise nas 32 respostas coletadas por meio do formulário (Apêndice A) divulgado entre alunos da UFERSA, profissionais da área de Tecnologia da Informação (TI) e redes sociais, que forneceu uma amostra diversificada de profissionais de diferentes áreas de atuação e níveis de experiência. A partir desses dados, foi possível mapear cenários comuns, identificar desafios frequentes no processo de implementação de testes, e sugerir melhorias ou práticas que auxiliem os desenvolvedores a superar tais obstáculos no contexto dos testes de software.

Figura 1 – Formulário de pesquisa - Área com maior experiência

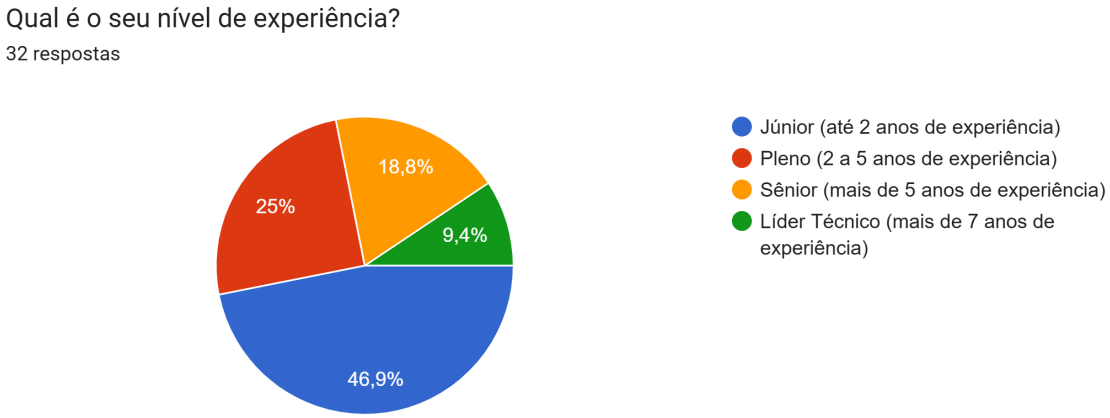


Fonte: Autoria própria (2024)

Tal análise foi realizada a partir do estudo das 32 respostas obtidas no formulário aplicado, começando pela identificação do perfil predominante entre os participantes dos testes. Como

mostrado na Figura 1, a maioria dos respondentes são profissionais da área de Quality Assurance (QA), desenvolvedores *full-stack* e programadores *front-end*.

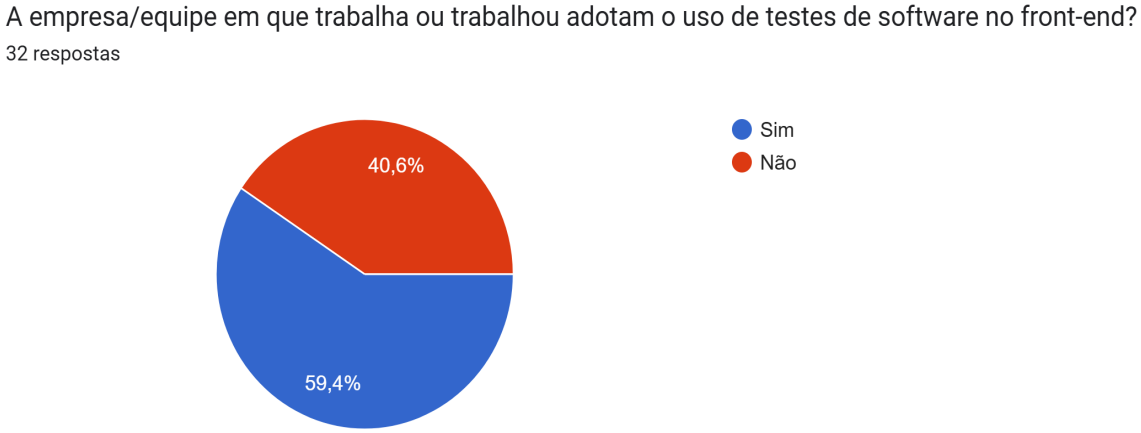
Figura 2 – Formulário de pesquisa - Perfil do participante



Fonte: Autoria própria (2024)

Com base na figura 2, Observa-se uma participação expressiva de profissionais ao nível Júnior, que representaram 46,9% dos participantes, enquanto uma parcela significativa de 25% foi composta por desenvolvedores Plenos. Essa distribuição indica um engajamento considerável de profissionais em início de carreira, juntamente com a contribuição de desenvolvedores mais experientes, enriquecendo a diversidade de perspectivas na pesquisa.

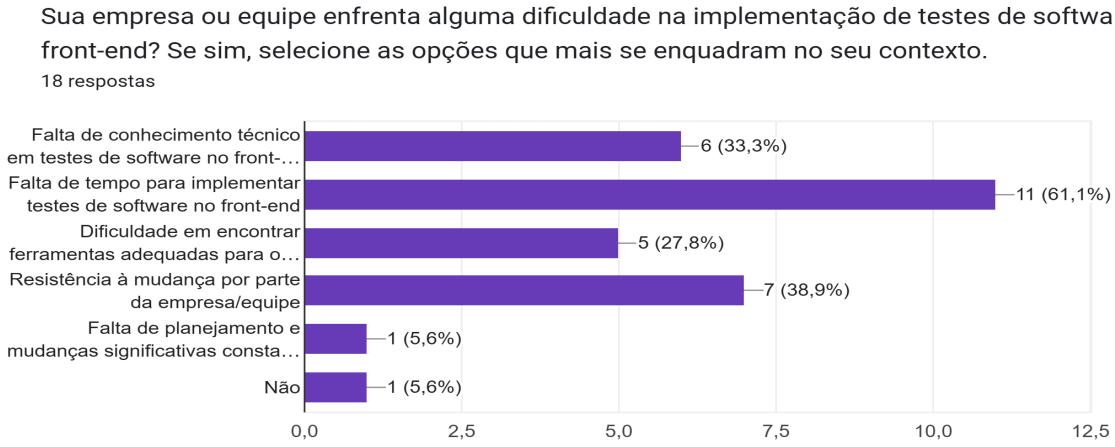
Figura 3 – Formulário de pesquisa - Adoção de testes de software



Fonte: Autoria própria (2024)

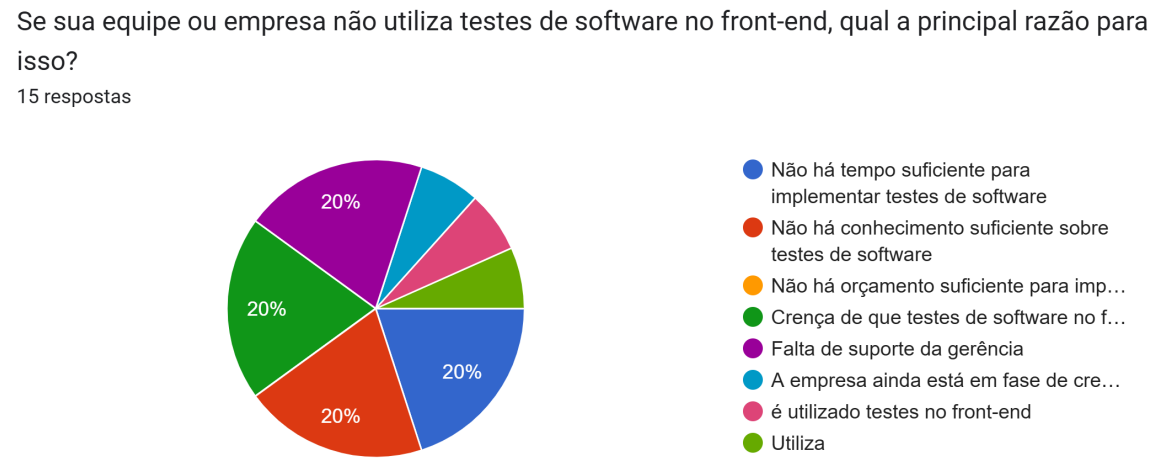
A pesquisa concentrou-se em mapear a adoção e as práticas de testes de software entre empresas e profissionais, buscando entender como e em que medida esses testes são incorporados nos projetos. Como pode ser observado na Figura 3, 59,4% dos participantes responderam que adotam o uso de testes no *front-end*. Embora seja um percentual relevante, é inferior ao esperado, considerando a importância dos testes de software para auxiliar na garantia de qualidade das aplicações.

Figura 4 – Formulário de pesquisa - Dificuldades das equipes com teste de software)



Fonte: Autoria própria (2024)

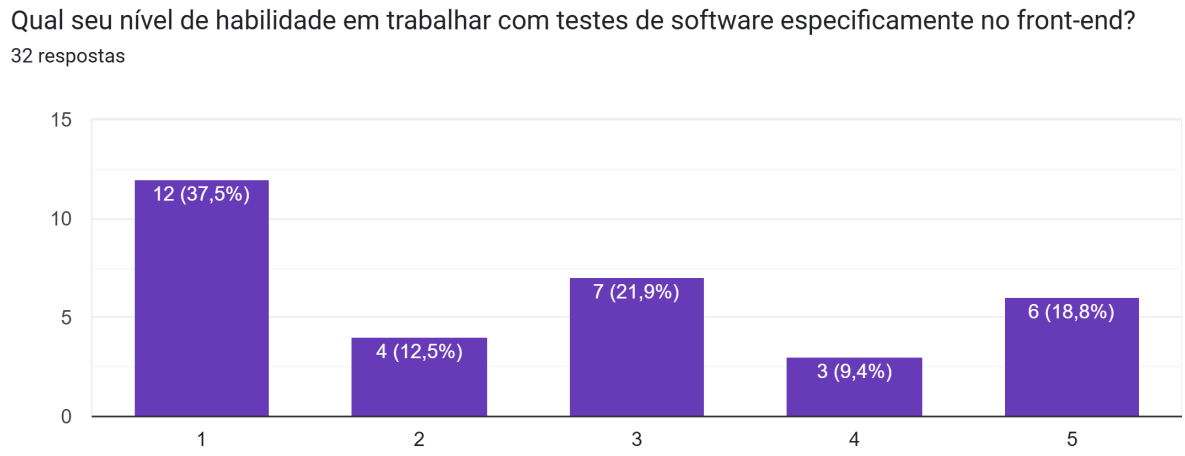
Figura 5 – Formulário de pesquisa - Dificuldades das equipes sem teste de software



Fonte: Autoria própria (2024)

Conforme mostrado nas Figuras 4 e 5, tanto as equipes que adotam testes de software quanto aquelas que ainda não os implementam enfrentam dificuldades devido à falta de conhecimento nessa área. Esse cenário ressalta a importância deste trabalho, que tem como objetivo fornecer uma ferramenta de aprendizado voltada para desenvolvedores, auxiliando-os na construção de habilidades essenciais em testes de software.

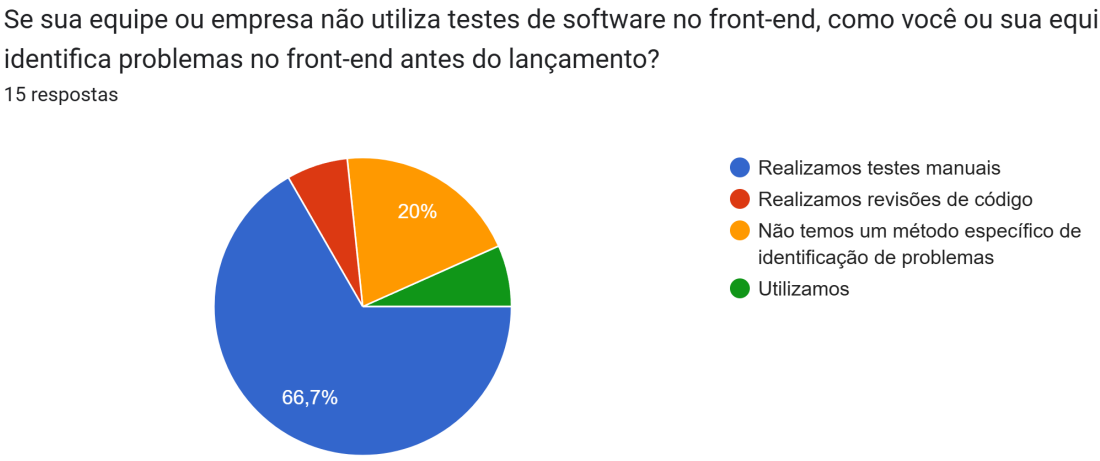
Figura 6 – Formulário de pesquisa - Habilidade em teste de software no *front-end*



Fonte: Autoria própria (2024)

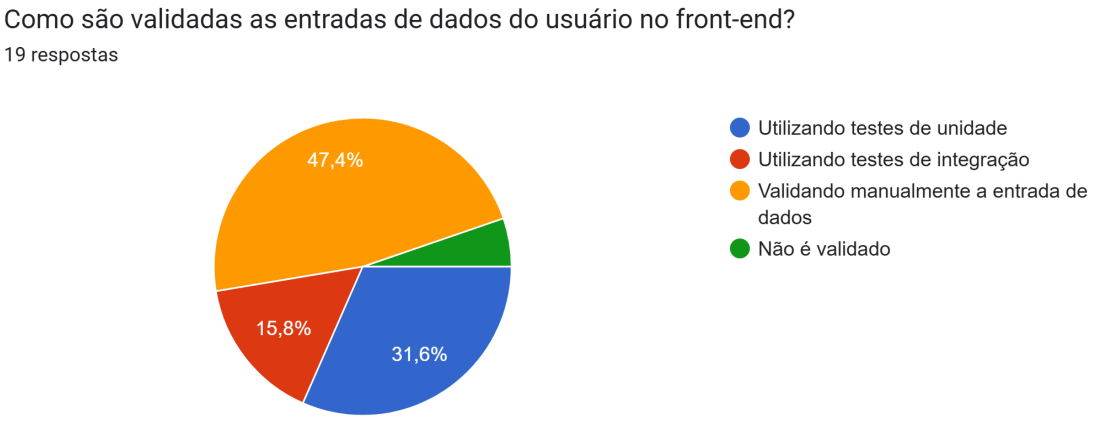
Na Figura 6, observa-se que uma grande parcela dos participantes possui apenas um conhecimento básico, evidenciando pouca habilidade com testes de software. Esse dado destaca a importância do guia de testes, que visa auxiliar desenvolvedores iniciantes a fortalecerem suas habilidades e adotarem melhores práticas no desenvolvimento *front-end*.

Figura 7 – Formulário de pesquisa - Aplicação de teste manuais (equipes sem teste de software)



Fonte: Autoria própria (2024)

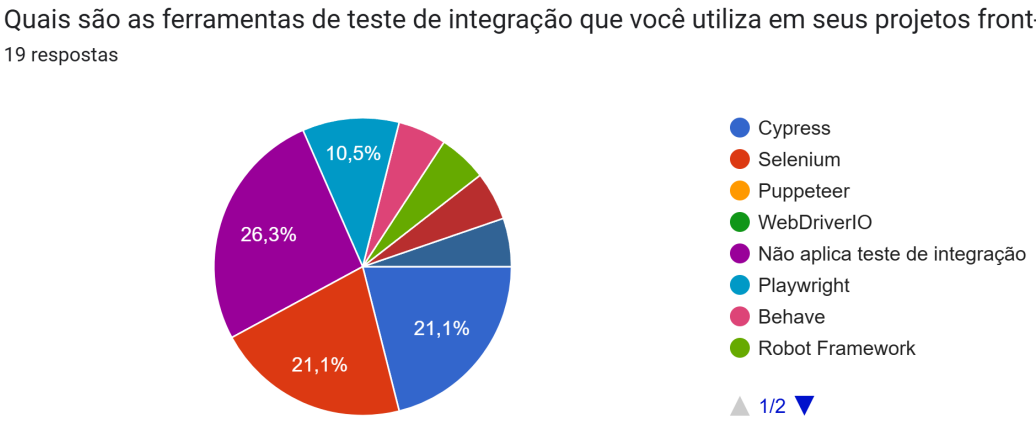
Figura 8 – Formulário de Pesquisa - Aplicação de testes manuais (equipes com teste de software)



Fonte: Autoria própria (2024)

Com base nas Figuras 7 e 8, observa-se uma tendência no uso de testes manuais, tanto em equipes que adotam testes de software quanto naquelas que não os utilizam regularmente. Esse resultado contribuiu para a decisão de focar o guia em exemplos práticos de testes automatizados, visando incentivar a adoção de práticas mais eficientes e consistentes.

Figura 9 – Formulário de Pesquisa - Ferramentas utilizadas pelos participantes



Fonte: Autoria própria (2024)

Para definir as ferramentas utilizadas no guia, foram avaliadas as tecnologias empregadas pelos participantes da pesquisa. Como mostrado na Figura 9, o Cypress[2] é amplamente utilizado, e, portanto, será adotado no guia. Além dele, o Jest[3] também será incluído, devido ao conhecimento prévio do autor sobre essa ferramenta. Essa escolha visa garantir que os leitores tenham acesso a ferramentas eficazes e amplamente reconhecidas no mercado.

5.2 Guia para testes Front-End

Para a elaboração do guia, foi adotada a plataforma *GitHub Pages*[4], possibilitando a disponibilização e distribuição do conteúdo de forma fácil e acessível para todos os interessados. Além de ser uma solução prática, o *GitHub Pages* permite que o guia seja hospedado gratuitamente e acessado por meio de um link, facilitando tanto a atualização contínua do conteúdo quanto a colaboração de outros desenvolvedores, caso o projeto seja aberto para contribuições.

O guia[5] foi desenvolvido para não apenas apresentar boas práticas no desenvolvimento de testes, mas também fornecer exemplos práticos e detalhados de sua aplicação em diferentes tipos de testes, como unitários, de integração e end-to-end (E2E). Além de abordar conceitos teóricos, o guia se concentra em demonstrar o uso prático das bibliotecas Jest e Cypress para a automação de testes em aplicações *front-end*, destacando suas funcionalidades e vantagens no

[2] Disponível em: https://www.cypress.io/
[3] Disponível em: https://jestjs.io/pt-BR/
[4] Disponível em: https://pages.github.com/
[5] Disponível em: https://rosluc.github.io/front_test_guide/

contexto de testes modernos.

Com isso, o guia oferece uma abordagem completa, que vai desde as melhores práticas até a aplicação prática das ferramentas, facilitando a adoção de uma cultura de testes robusta e eficiente no desenvolvimento *front-end*.

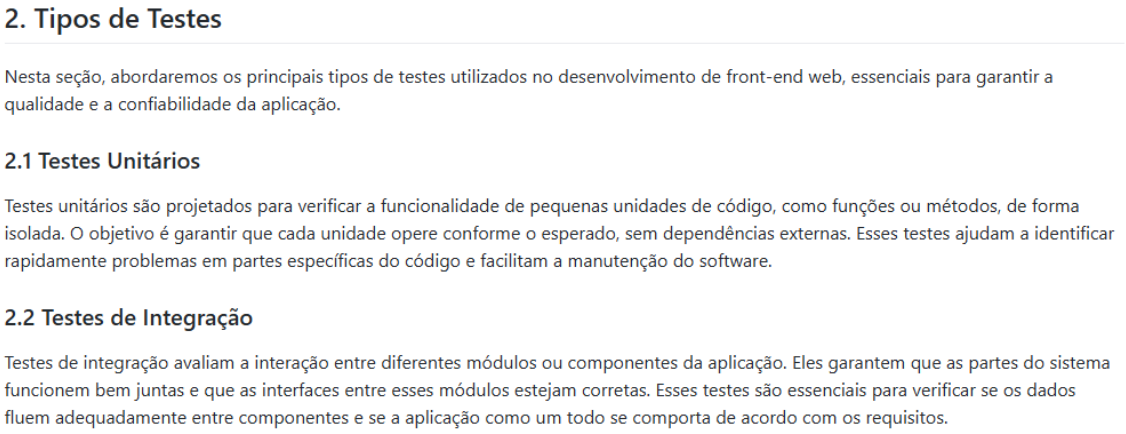
Figura 10 – Sumário do guia



Fonte: Autoria própria (2024)

Como apresentado na Figura 10, o guia é estruturado em cinco seções principais, cada uma com uma função específica para orientar o aprendizado em testes de software. A primeira seção, Introdução, descreve o propósito do guia, destacando a sua importância como recurso educacional, e identifica o público-alvo, que inclui desenvolvedores iniciantes e profissionais que desejam aprimorar suas habilidades em testes de software. Esta seção busca contextualizar o leitor sobre os objetivos do guia e os conhecimentos prévios recomendados para melhor aproveitamento do conteúdo.

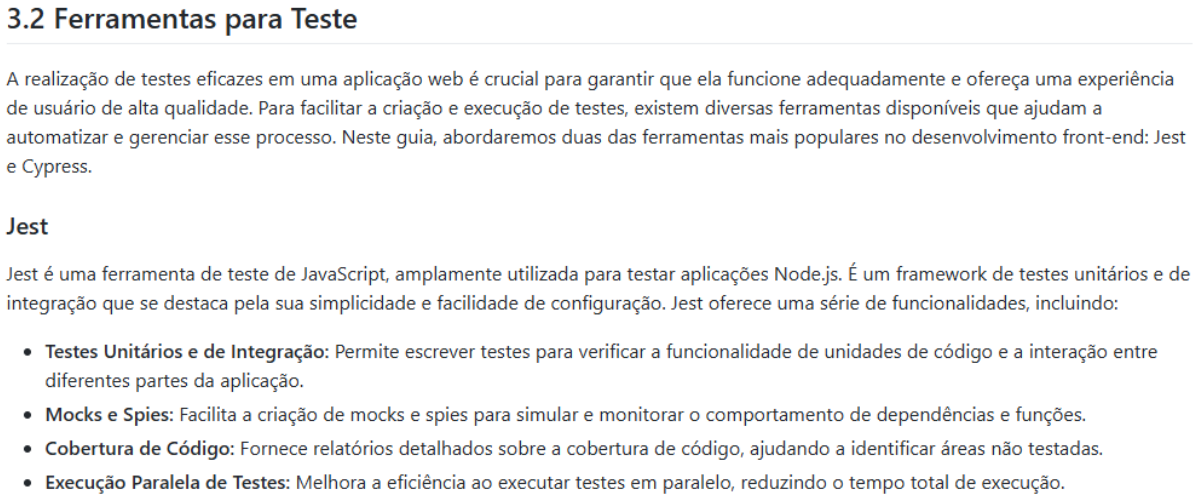
Figura 11 – Segunda seção do guia



Fonte: Autoria própria (2024)

Na sequência, como pode ser observado na Figura 11, discutimos os diferentes tipos de testes que são fundamentais para o desenvolvimento *front-end*. Esta seção abrange testes unitários, de integração e testes E2E) entre outros. Cada um desses tipos de teste desempenha um papel crucial na garantia da qualidade do software.

Figura 12 – Terceira seção do guia



Fonte: Autoria própria (2024)

Na Figura 12, observa-se que a terceira seção é dedicada à configuração das ferramentas utilizadas no guia. Essas ferramentas foram escolhidas com base na familiaridade do autor e nas recomendações apresentadas na pesquisa discutida anteriormente na seção 5.1. Nesta parte, detalhamos o processo de instalação e configuração, conforme a documentação oficial das bibliotecas empregadas, como Jest e Cypress. O objetivo é assegurar que os leitores tenham um

ambiente de testes devidamente configurado e pronto para uso, facilitando a implementação dos testes.

Figura 13 – Quarta seção do guia



Fonte: Autoria própria (2024)

Na Figura 13, a quarta seção apresenta boas práticas para o desenvolvimento de testes, enfocando estratégias que garantem a qualidade e a eficiência dos testes automatizados. Nesta seção, discutimos a importância de manter a clareza e a legibilidade dos testes, entre outros aspectos essenciais para facilitar a compreensão e a manutenção dos mesmos.

Figura 14 – Quinta seção do guia



5. Exemplos de Testes

Neste topico, será abordado testes na aplicação front-end.

5.1 Testes Unitários

Para iniciar os testes unitários, é necessário definir o que é uma unidade em seu projeto. Considerando que nossa aplicação alvo é um front-end em Next.js, podemos definir como unidades as funções utilitárias, que geralmente são utilizadas para manipular dados na aplicação, além dos hooks e contexts.

Em nossa aplicação alvo, o Me Contrata, temos as seguintes funções utilitárias no módulo de listagem de serviços.

```
import { FilterType } from "@/types/filter-types";
import { PriorityTypes } from "@/types/priority-types";

export function getCategoryByType(type: FilterType) {

  switch (type) {
    case FilterType.PROGRAMMING:
      return 'PROGRAMMING';
    case FilterType.DESIGN:
      return 'DESIGN';
    default:
      return "";
  }
}

export function getFieldByPriority(priority: PriorityTypes) {

  switch (priority) {

    case PriorityTypes.NEWS:
      return { field: "created_at", order: "ASC" };
    case PriorityTypes.BIGGEST_PRICE:
```

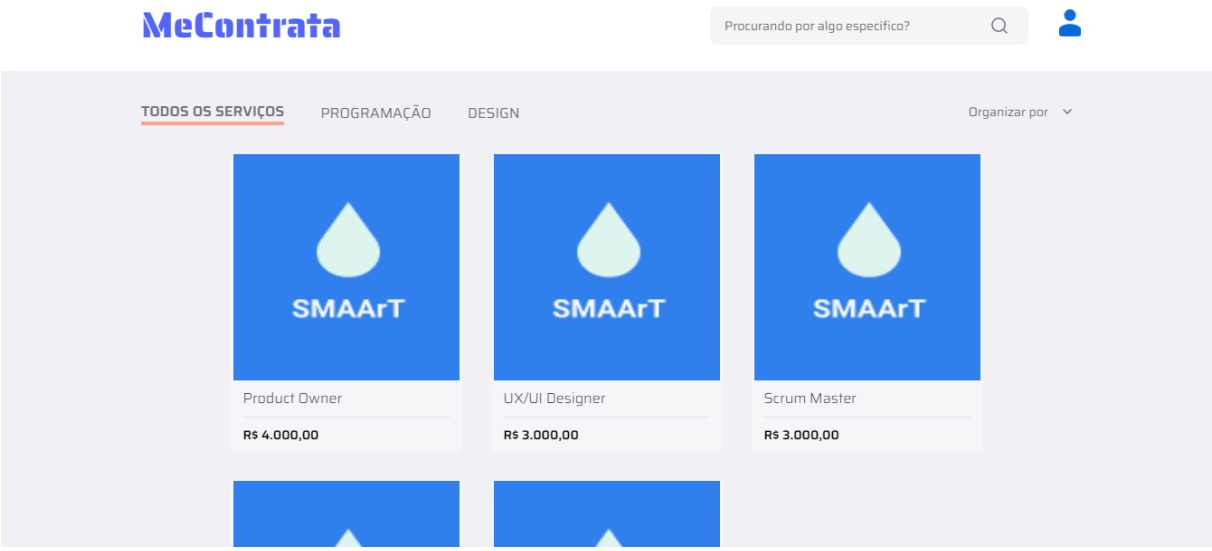
Fonte: Autoria própria (2024)

Finalmente, na Figura 14 temos a última seção que apresenta exemplos práticos que ilustram a construção de testes unitários, de integração e E2E. Esses exemplos são projetados para demonstrar de forma prática como aplicar os conceitos discutidos ao longo do guia, utilizando as ferramentas que foram previamente configuradas. Ao longo desta seção, os leitores poderão acompanhar passo a passo o processo de criação dos testes, compreendendo as boas práticas de cada tipo de teste. Essa abordagem prática visa não apenas solidificar o aprendizado, mas também capacitar os desenvolvedores a implementarem testes de forma eficaz em seus próprios projetos, reforçando a importância da qualidade no desenvolvimento de software.

Para a elaboração dos exemplos práticos dos testes automatizados, foi adotado uma aplicação específica que serviu como ambiente de estudo. Esta aplicação foi adotada com o objetivo principal de demonstrar de maneira prática como os testes podem ser elaborados e aplicados no contexto do desenvolvimento *front-end* Web. Ao proporcionar exemplos concretos de implementação de testes, pretende-se facilitar a compreensão dos leitores, permitindo-lhes visualizar diretamente como essas técnicas podem ser aplicadas em situações reais de desenvolvimento de

software.

Figura 15 – Tela do sistema MeContrata



Fonte: Autoria própria (2024)

Na Figura 15, temos o sistema MeContrata, um software desenvolvido pelo autor em colaboração com outro aluno do curso de Engenharia de Software da UFERSA, que tem como objetivo ser um meio de comunicação entre empresas e profissionais da área de TI. Ao contrário das abordagens convencionais, os profissionais divulgam seus serviços e as empresas podem encontrá-los na plataforma, manifestando interesse em contratar o profissional ou o serviço oferecido.

Conforme mencionado anteriormente, o guia usa as ferramentas Jest e Cypress, amplamente utilizadas no ecossistema Node.js. O Jest é uma biblioteca versátil, aplicada tanto em testes de *back-end* quanto de *front-end*, sendo uma escolha popular por sua simplicidade e eficácia na execução de testes unitários e de integração. Já o Cypress tem um foco mais específico em aplicações Web, sendo uma ferramenta poderosa para testes E2E, que simulam o comportamento do usuário real no navegador, garantindo a validação completa das interfaces e fluxos de uma aplicação.

Ambas as ferramentas foram escolhidas por sua robustez e por contarem com uma comunidade ativa, além de oferecerem uma boa integração com projetos em Node.js, tornando-as ideais para a automação de testes em projetos front-end desenvolvidos com NextJS. A combinação de Jest e Cypress no guia visa proporcionar uma cobertura abrangente de testes, desde os componentes isolados até a experiência completa do usuário na aplicação Web.

Com isso, o guia se concentra em apresentar exemplos práticos baseados na pirâmide de testes, abordando os três principais níveis: testes unitários, testes de integração e testes E2E. Ele explora como cada ferramenta pode ser utilizada eficazmente para cada tipo de teste, detalhando os alvos específicos que devem ser verificados.

No caso de uma aplicação Next.js, são destacados diferentes aspectos que precisam ser testados, como funções utilitárias, componentes, requisições à API e ações realizadas na interface do usuário. O guia mostra como os testes unitários são ideais para validar funções e componentes isoladamente, enquanto os testes de integração verificam a interação entre diferentes partes do sistema. Já os testes E2E, com o uso do Cypress, garantem que os fluxos completos da aplicação, incluindo interações de usuário e navegações, estejam funcionando conforme o esperado.

Dado tudo isso, o guia se configura como uma ferramenta de introdução aos testes *front-end*, oferecendo aos desenvolvedores um ponto de partida para compreender e aplicar as principais práticas de testes no desenvolvimento Web. Ao fornecer explicações claras, exemplos práticos e a utilização de ferramentas amplamente adotadas como Jest e Cypress, o guia serve como um recurso valioso para aqueles que estão iniciando no segmento de testes automatizados.

Além de fornecer as bases necessárias, o guia também incentiva o aprendizado contínuo, permitindo que os desenvolvedores possam evoluir e expandir seu conhecimento à medida que ganham mais experiência com a implementação de testes em projetos reais. Isso os prepara para enfrentar desafios mais complexos no futuro e desenvolver uma mentalidade de qualidade e robustez em seus códigos.

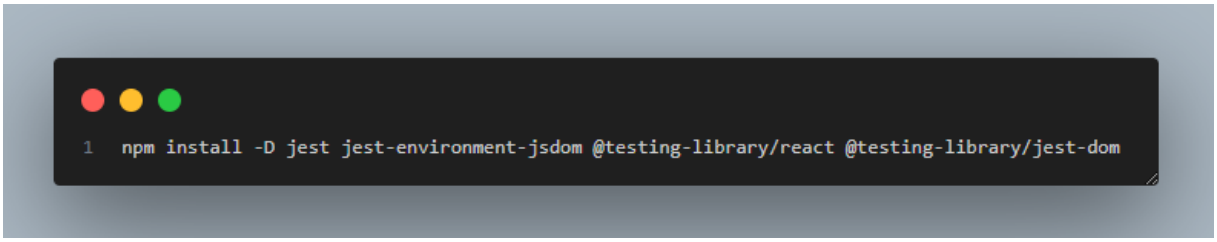
Por se tratar de um projeto disponibilizado no GitHub, todos os interessados poderão contribuir para o seu aprimoramento. Isso cria uma oportunidade para que a comunidade de desenvolvedores participe ativamente na manutenção e atualização do guia, garantindo que ele permaneça alinhado com as práticas mais recentes do mercado de trabalho. Além disso, a plataforma permite que novos *insights* e informações sejam incorporados, o que pode enriquecer o conteúdo com a experiência de outros profissionais, fortalecendo a ferramenta como um recurso colaborativo e em constante evolução.

Com essa abordagem, o guia não apenas oferece um ponto de partida sólido para novos desenvolvedores, mas também se torna um projeto vivo, onde contribuições externas podem aprimorar continuamente sua qualidade e relevância no cenário do desenvolvimento *front-end*.

5.3 Aplicação do guia para testes *front-end*

Dado o sistema Me Contrata[6], o guia será aplicado nos módulos e componentes que ainda não foram utilizados nos exemplos práticos do guia, a fim de exemplificar o passo a passo do seu uso. Partindo do ponto que o interessado no guia realizou a leitura e compreensão dos tipos de testes e boas práticas presentes nas seções 2 e 4 do guia, partimos para aplicação dos testes automatizado no Me Contrata.

Figura 16 – Guia: Dependências de pacotes para o Jest



Fonte: Autoria própria (2024)

Partindo para a configuração do ambiente com Jest presente seção 3 do guia (Figura 16), serão instaladas as dependências necessárias, conforme informado no guia. Isso inclui o pacote do Jest e suas bibliotecas auxiliares, essenciais para o funcionamento correto no Next.js.

Figura 17 – Guia: Arquivo de definições do Jest



Fonte: Autoria própria (2024)

[6] Disponível em: https://github.com/RosLuc/me-contrata-front

Como pode ser visto na Figura 17, a próxima etapa envolve a configuração do arquivo de definições do Jest. Essa configuração é fundamental para garantir que o Jest funcione corretamente no contexto do projeto. Na linha 6, especificamos o diretório do projeto, o que permite que o Jest saiba onde localizar os arquivos relevantes para os testes. Essa informação é crucial, pois o Jest precisa identificar onde estão localizados os componentes e módulos que serão testados.

Em seguida, na linha 8, definimos a configuração na constante apropriada. Essa configuração pode incluir várias opções, como o ambiente de teste, as transformações necessárias para os arquivos e as extensões a serem consideradas. Seguindo os passos descritos no guia, garantimos que todas as configurações estejam segundo as melhores práticas para um funcionamento ideal do Jest no Next.js. Esse cuidado na configuração não só ajuda a evitar erros durante a execução dos testes, mas também assegura que a estrutura do projeto esteja alinhada com os requisitos do Jest, permitindo uma integração mais fluida e eficaz.

Figura 18 – Guia: Exemplo de componente a ser testado

```tsx
"use client";

import styles from "./styles.module.css";
import { useFilter } from "@/hooks/useFilter";
import { FilterType } from "@/types/filter-types";

export function FilterByType() {
    const { type, setType } = useFilter();

    const handleChangeType = (value: FilterType) => {
        setType(value);
    };
    return (
        <ul className={styles.filter_list}>
            <li
                className={type === FilterType.ALL ? styles.selected : ""}
                onClick={() => handleChangeType(FilterType.ALL)}
            >
                Todos os serviços
            </li>
            <li
                className={type === FilterType.PROGRAMMING ? styles.selected : ""}
                onClick={() => handleChangeType(FilterType.PROGRAMMING)}
            >
                Programação
            </li>
            <li
                className={type === FilterType.DESIGN ? styles.selected : ""}
                onClick={() => handleChangeType(FilterType.DESIGN)}
            >
                Design
            </li>
        </ul>
    );
}
```

Fonte: Autoria própria (2024)

Com base nos conceitos do guia focaremos em aplicar testes em um componente da aplicação Me Contrata, que por ser feita em Next.js, garantir o funcionamento dos componentes é de grande importância. Dessa forma, como ilustrado na Figura 18, apresentamos o componente que será submetido a um teste automatizado, visando validar seu funcionamento de maneira eficaz.

Figura 19 – Guia: Dependências necessárias para o Jest no *front-end*

```tsx
// FilterByType.test.tsx

import React from 'react';
import { render, fireEvent } from '@testing-library/react';
import { FilterByType } from '../index';
import { useFilter } from '../../../hooks/useFilter';
import { FilterType } from '../../../types/filter-types';
import '@testing-library/jest-dom';

jest.mock('../../../hooks/useFilter');

describe('FilterByType', () => {

  const mockSetType = jest.fn();

  beforeEach(() => {
    (useFilter as jest.Mock).mockReturnValue({
      type: FilterType.ALL,
      setType: mockSetType,
    });
  });
```

Fonte: Autoria própria (2024)

Com base nos exemplos e conceitos abordados no guia, como pode ser visto na Figura 19, a configuração inicial do arquivo de teste inclui as importações necessárias, além das definições de bibliotecas e funções emuladas do sistema fundamentais para a elaboração dos testes. Essa configuração cria um ambiente controlado, permitindo que os testes sejam realizados de forma eficiente e precisa.

Após essa configuração inicial, a próxima etapa é começar a escrever os casos de teste para o componente. Nesta fase, é crucial validar tanto as entradas quanto as saídas da execução do componente, assegurando que ele se comporta conforme o esperado em diferentes cenários.

Figura 20 – Guia: Exemplo de teste construídos com o Jest

```
it('should call setType with correct value when an option is clicked', () => {

    const { getByText } = render(<FilterByType />);

    fireEvent.click(getByText('Todos os serviços'));
    expect(mockSetType).toHaveBeenCalledWith(FilterType.ALL);

    fireEvent.click(getByText('Programação'));
    expect(mockSetType).toHaveBeenCalledWith(FilterType.PROGRAMMING);

    fireEvent.click(getByText('Design'));
    expect(mockSetType).toHaveBeenCalledWith(FilterType.DESIGN);
  });
```

Fonte: Autoria própria (2024)

Pode-se observar na Figura 20 um exemplo de teste construído para esse componente, utilizando os conceitos de simulação apresentados no guia. Neste exemplo, é validada a interação do componente em resposta a eventos de clique no sistema.

O teste simula as ações do usuário, permitindo verificar se o componente se comporta corretamente quando interage com diferentes elementos da interface. Através da simulação de cliques, podemos assegurar que as funções associadas ao componente estão sendo acionadas corretamente e que a lógica de manipulação de estados e propriedades está funcionando como esperado.

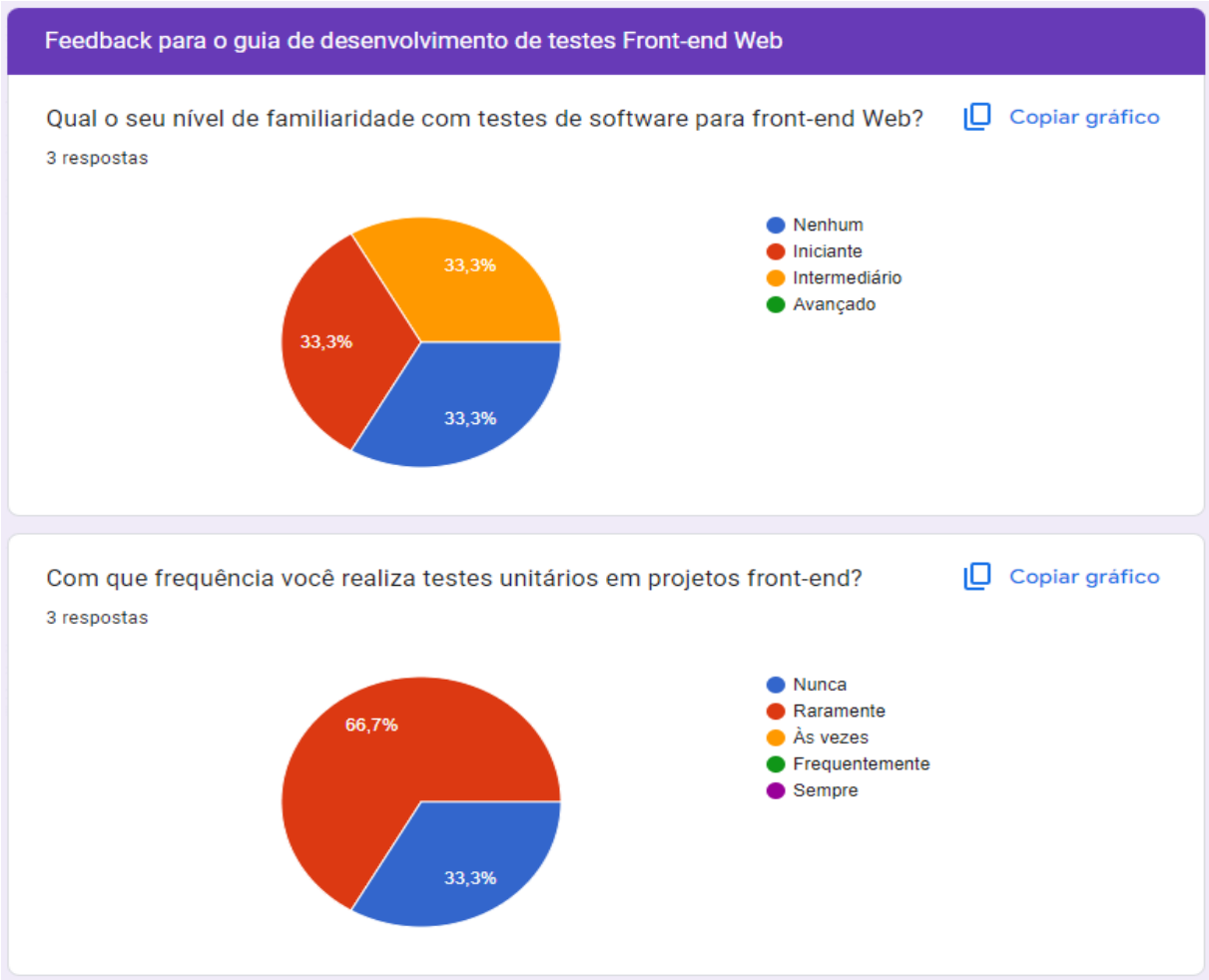
Logo, com base no guia, é possível transitar de uma aplicação sem testes automatizados para a implementação dos testes iniciais, sempre focando em seguir as boas práticas abordadas. Esse processo pode ser realizado em etapas, permitindo uma transição gradual e eficaz. Além de proporcionar novos conhecimentos e dicas para quem está começando na prática de testes.

5.4 Pesquisa de *feedback*

O guia foi disponibilizado a uma equipe de três desenvolvedores que atuam em uma empresa responsável pela manutenção e evolução de seus sistemas. Esses desenvolvedores trabalham na mesma organização que o autor deste estudo. Diante do interesse demonstrado por esses colaboradores em relação à testes de software, decidiu-se oferecer o guia a eles e coletar seus *feedbacks*. Essa iniciativa visa aprimorar a aplicação dos conceitos abordados no guia e

avaliar sua efetividade no contexto prático da equipe. O formulário (Apêndice B) teve como objetivo coletar *feedbacks* da equipe que utilizou o guia, permitindo avaliar a eficácia do material e verificar se os objetivos propostos foram devidamente atendidos.

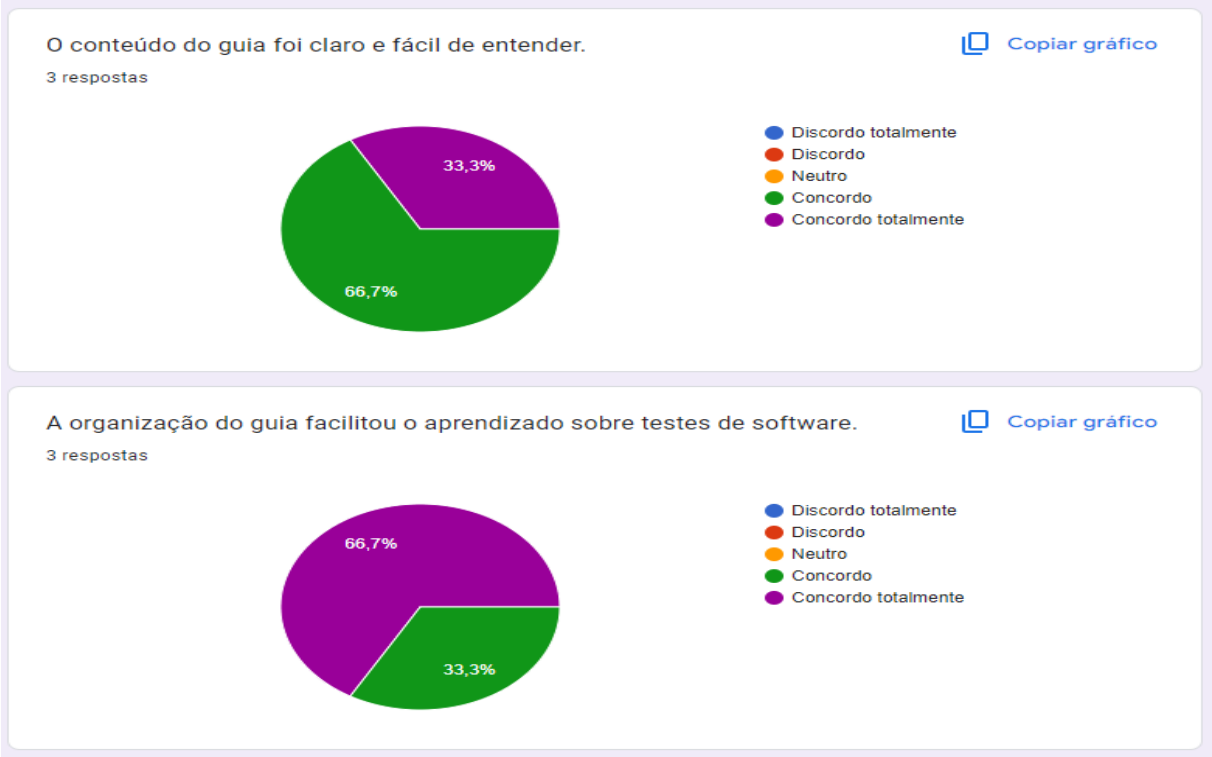
Figura 21 – Perfil dos desenvolvedores com relação à prática de teste



Fonte: Autoria própria (2024)

Como pode ser observado na Figura 21, o time apresentava um conhecimento limitado e pouca prática em testes unitários em projetos *front-end*. Essa constatação evidencia a necessidade do guia, cujo principal objetivo é fornecer suporte e orientação a desenvolvedores que desejam aprimorar suas habilidades nessa área específica.

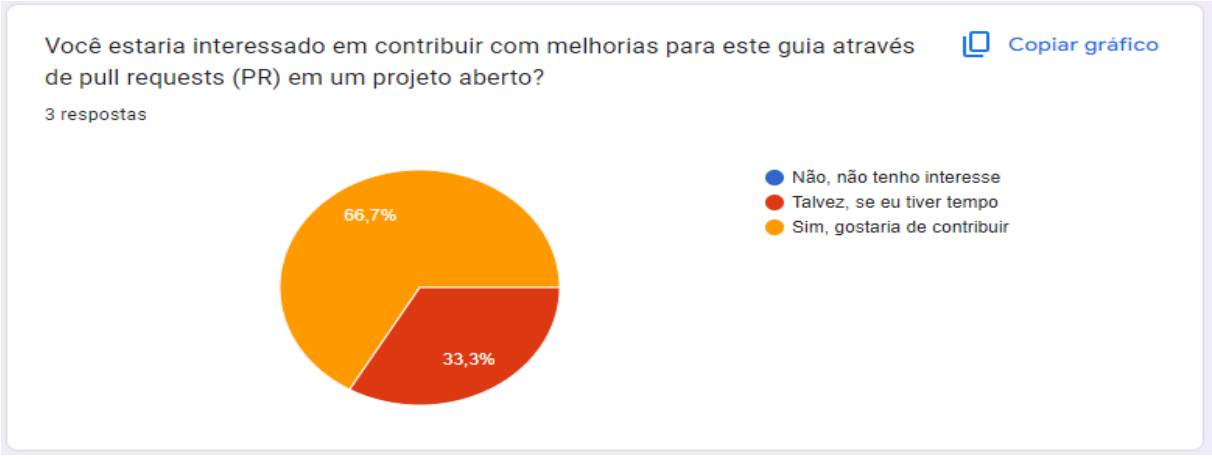
Figura 22 – Perfil dos desenvolvedores sobre o conteúdo e organização do guia



Fonte: Autoria própria (2024)

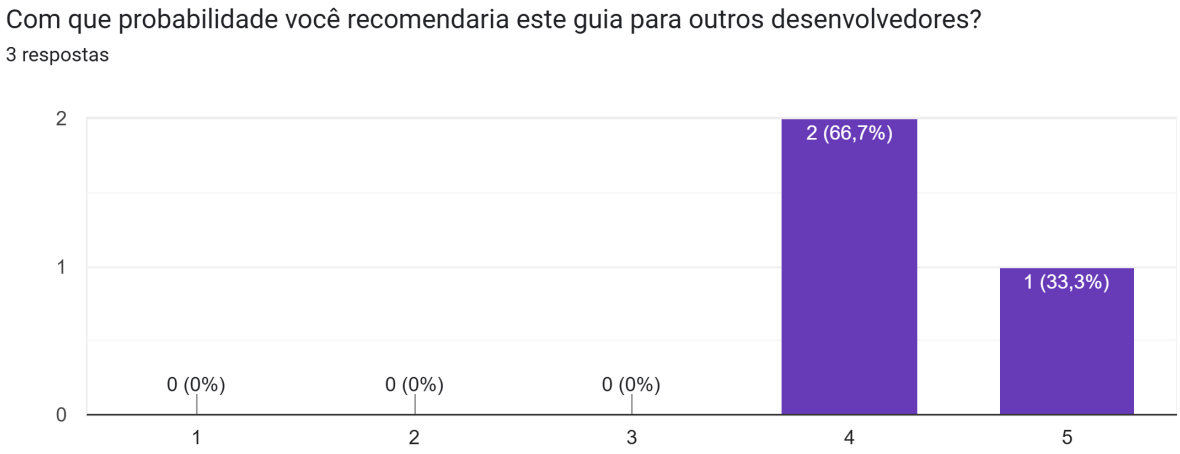
Na Figura 22, pode-se observar que os participantes demonstraram uma boa afinidade com o material do guia e sua estrutura. Essa receptividade indica que o conteúdo foi bem recebido e a organização das informações facilitou a compreensão dos conceitos abordados, contribuindo para um aprendizado mais eficaz.

Figura 23 – Interesse dos participantes em contribuir com o guia



Fonte: Autoria própria (2024)

Figura 24 – Interesse dos participantes em compartilhar o guia



Fonte: Autoria própria (2024)

Observa-se na Figura 23 que o trabalho conseguiu atrair o interesse dos participantes em contribuir, evidenciando seu potencial como um material aberto e em constante evolução. Além disso, observa-se na Figura 24 que os participantes demonstraram interesse em compartilhar o guia, contribuindo assim para sua divulgação e ampliação do alcance.

Essa disposição para disseminar o material é um indicativo da relevância e utilidade do guia no contexto da equipe, podendo potencialmente beneficiar outros profissionais na adoção de práticas de testes de software. Essa dinâmica de colaboração sugere que o guia não apenas serve como um recurso estático, mas também como uma plataforma interativa que pode ser continuamente aprimorada com novas ideias e *feedbacks*, promovendo um aprendizado colaborativo e enriquecedor.

6 CONCLUSÕES E TRABALHOS FUTUROS

Ao final deste projeto, foi elaborado um guia introdutório para desenvolvedores *front-end* Web focado no segmento de testes de software. Esse guia se torna uma ferramenta valiosa de aprendizado, oferecendo um ponto de partida claro e acessível para iniciantes na área. Além disso, trata-se de um projeto vivo, passível de ser continuamente aprimorado e mantido pela comunidade. Dessa forma, o conhecimento poderá ser centralizado e facilmente acessado por desenvolvedores, estudantes e entusiastas que desejam aprofundar seus conhecimentos em testes de software, garantindo uma evolução contínua e colaborativa na área.

Contudo, além de ser um projeto em constante evolução, no qual outras pessoas podem se tornar colaboradoras, este trabalho apresenta diversas possibilidades de desenvolvimento futuro. É essencial expandir o guia com foco em aspectos fundamentais como segurança e desempenho. Essas áreas são cruciais no desenvolvimento de aplicações *front-end* e, portanto, merecem uma atenção especial para garantir que o guia aborde de maneira abrangente os desafios e as melhores práticas para criar aplicações não apenas funcionais, mas também seguras e eficientes. Essa expansão proporcionará uma base ainda mais sólida para desenvolvedores que desejam se aprofundar nessas áreas críticas do desenvolvimento Web.

Com a facilitação do projeto ser *open-source* e a colaboração de outros desenvolvedores, será possível ampliar o guia para incluir exemplos com uma variedade maior de ferramentas de automação, assim como outros *frameworks* de desenvolvimento Web. Dessa forma, o guia poderá evoluir de um recurso inicial para um verdadeiro guia “definitivo” sobre testes Web. Essa evolução permitirá que desenvolvedores de todos os níveis encontrem diretrizes práticas e abrangentes, contribuindo para uma comunidade mais informada e preparada para enfrentar os desafios do desenvolvimento *front-end*.

Para validar a eficácia do guia, é pertinente realização de um trabalho futuro com objetivo de realizar um estudo envolvendo um número maior de desenvolvedores. Essa abordagem permitiria acompanhar a utilização do guia de forma sistemática, possibilitando uma compreensão mais abrangente de seu impacto enquanto ferramenta de ensino. A avaliação contínua das práticas e resultados obtidos pode fornecer *insights* valiosos para futuras melhorias e adaptações do material.

REFERÊNCIAS

ALBERTO, J. *et al.* Automação de testes: Um estudo de caso. In: **Information Systems in Latin America**. [S.l.]: AIS eLibrary, 2020.

AZZAM, F. *et al.* Evaluation for web gui automation testing tool-experiment. In: IEEE. **2022 International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)**. [S.l.], 2022. p. 549–554.

BERNARDO, P. C.; KON, F. A importância dos testes automatizados. **Engenharia de Software Magazine**, v. 1, n. 3, p. 54–57, 2008.

BORJESSON, E.; FELDT, R. Automated system testing using visual gui testing tools: A comparative study in industry. In: IEEE. **2012 IEEE Fifth International Conference on Software Testing, Verification and Validation**. [S.l.], 2012. p. 350–359.

BUENO, C. d. F. dos S.; CAMPELO, G. B. Qualidade de software. 2011.

CAMPOPIANO, G. T. **Importância de testes unitários no desenvolvimento web**. 2022. Trabalho de Conclusão de Curso (graduação) - Faculdade de Tecnologia de São Paulo, Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas.

DONATO, M. **Proposição de um Guia de Automação de Testes de Software para o TCE-MT: um Estudo de Caso**. Tese (Doutorado) — Universidade Federal de Mato Grosso, 2023.

IJAZ, T.; ANDLIB, F. Impact of usability on non-technical users: Usability testing through websites. In: IEEE. **2014 national software engineering conference**. [S.l.], 2014. p. 37–42.

JÚNIOR, A. A. Processo de teste de software: uma descrição com a perspectiva da qualidade de software. Universidade Federal de Minas Gerais, 2011.

KINSBRUNER, E.; BAHMUTOV, G. [S.l.: s.n.], 2022.

LIU, K. *et al.* Design and development of management information system for research project process based on front-end and back-end separation. In: IEEE. **2017 International Conference on Computing Intelligence and Information System (CIIS)**. [S.l.], 2017. p. 338–342.

LORENÇON, B. da S.; GIBERTONI, D. Investigar os procedimentos de ux durante o desenvolvimento de websites. **Revista Interface Tecnológica**, v. 18, n. 2, p. 253–263, 2021.

MALDONADO, J. C. *et al.* Introducao ao teste de software (versão 2004-01). 2004.

MOSER, G. *et al.* Teaching software quality assurance with gamification and continuous feedback techniques. In: IEEE. **2021 IEEE Global Engineering Education Conference (EDUCON)**. [S.l.], 2021. p. 505–509.

MOURA, T. S. d. **Automação de testes em aplicações web utilizando uma abordagem ad-hoc.** 2021. Trabalho de Conclusão de Curso (graduação) - Universidade Federal de Campina Grande, Curso de Ciência da Computação.

NASS, M.; ALÉGROTH, E.; FELDT, R. Why many challenges with gui test automation (will) remain. **Information and Software Technology**, Elsevier, v. 138, p. 106625, 2021.

PACHECO, J.; GARBATOV, S.; GOULÃO, M. Improving collaboration efficiency between ux/ui designers and developers in a low-code platform. In: IEEE. **2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)**. [S.l.], 2021. p. 138–147.

PRESSMAN, R.; MAXIM, B. **Engenharia de Software-8ª Edição**. [S.l.]: McGraw Hill Brasil, 2016.

SAMPAIO, A. d. C. R. **Criação de um guia de teste de software a partir da análise de repositórios do Github**. 2022.

SOMMERVILLE, I. **Engenharia de Software**. [S.l.]: São Paulo: Pearson Prentice Hall, 2011. v. 9. ed.

THOORIQOH, H. A.; ANNISA, T. N.; YUHANA, U. L. Selenium framework for web automation testing: A systematic literature review. **Jurnal Ilmiah Teknologi Informasi**, Sepuluh Nopember Institute of Technology, v. 19, n. 2, p. 65–76, 2021.

WONG, W. Y.; YU, S. W.; TOO, C. W. A systematic approach to software quality assurance: the relationship of project activities within project life cycle and system development life cycle. In: IEEE. **2018 IEEE Conference on Systems, Process and Control (ICSPC)**. [S.l.], 2018. p. 123–128.

APÊNDICE A – PESQUISA SOBRE O TESTE DE SOFTWARE NO *FRONT-END*

Pesquisa Sobre o Teste de Software no Front-end

O *front-end* é uma das partes importantes de um sistema web ou móvel, pois trata-se do contato direto com o usuário. Logo, apresentar problemas ou inconsistência trara uma péssima experiencia para o usuário, o que fará com que abandone o sistema.

Esta pesquisa tem como intuito avaliar o nível de maturidade do teste de software no *front-end*, onde ajudará a identificar quais são os principais desafios e obstáculos que as empresas e equipes enfrentam ao implementar testes de software no *front-end*, identificando quais são as práticas e abordagens utilizadas.

O participante poderá deixar o formulário a qualquer momento sem qualquer prejuízo. Também não serão fornecidas quaisquer compensações financeiras aos participantes.

Informações de contato dos pesquisadores:

Rosendo Lucas Santana da Costa: rosendo.costa@alunos.ufersa.edu.br

Alysson Filgueira Milanez: alysson.milanez@ufersa.edu.br

* Indica uma pergunta obrigatória

1. Você concorda com os termos acima e deseja participar do presente estudo? *

   Marcar apenas uma oval.

   ◯ Sim *Pular para a pergunta 2*

   ◯ Não

Pesquisa Sobre o Teste de Software no Front-end

2. Em qual área o possui mais experiência? *

Marcar apenas uma oval.

- ◯ Front-end
- ◯ Back-end
- ◯ FullStack
- ◯ DevOps
- ◯ QA
- ◯ Outro: _____

3. Qual é o seu nível de experiência? *

Marcar apenas uma oval.

- ◯ Júnior (até 2 anos de experiência)
- ◯ Pleno (2 a 5 anos de experiência)
- ◯ Sênior (mais de 5 anos de experiência)
- ◯ Líder Técnico (mais de 7 anos de experiência)

4. Quais são as suas principais habilidades técnicas em desenvolvimento front-end?

Marque todas que se aplicam.

- ☐ HTML
- ☐ CSS
- ☐ JavaScript
- ☐ TypeScript
- ☐ React
- ☐ Angular
- ☐ Vue.js
- ☐ Bootstrap
- ☐ Material UI
- ☐ Tailwind CSS
- ☐ Outro: _____

5. Qual seu nível de habilidade em trabalhar com testes de software? *

Marcar apenas uma oval.

	1	2	3	4	5	
Inici	◯	◯	◯	◯	◯	Avançado

6. Qual seu nível de habilidade em trabalhar com testes de software especificamente no front-end? *

Marcar apenas uma oval.

	1	2	3	4	5	
Inici	◯	◯	◯	◯	◯	Avançado

7. A empresa/equipe em que trabalha ou trabalhou adotam o uso de testes de software no front-end? *

Marcar apenas uma oval.

- ◯ Sim *Pular para a pergunta 8*
- ◯ Não *Pular para a pergunta 19*

Pesquisa Sobre o Teste de Software no Front-end

8. Qual é o objetivo principal dos testes de software no front-end da sua empresa ou equipe? *

Marcar apenas uma oval.

- ◯ Garantir a qualidade do software
- ◯ Identificar e corrigir erros antes do lançamento
- ◯ Melhorar a experiência do usuário
- ◯ Outro: _____

9. Quais são os tipos de testes de software no front-end que são realizados na sua empresa ou equipe *

Marque todas que se aplicam.

- [ ] Testes unitários
- [ ] Testes de integração
- [ ] Testes de segurança
- [ ] Testes de regressão
- [ ] Testes de desempenho
- [ ] Testes de usabilidade
- [ ] Teste de compatibilidade
- [ ] Outro: _____

10. Como são validadas as entradas de dados do usuário no front-end? *

Marcar apenas uma oval.

- ◯ Utilizando testes de unidade
- ◯ Utilizando testes de integração
- ◯ Validando manualmente a entrada de dados
- ◯ Não é validado

11. Quais são as ferramentas de teste de unidade que você utiliza em seus projetos front-end? *

Marcar apenas uma oval.

- ◯ Jest
- ◯ Mocha
- ◯ Jasmine
- ◯ QUnit
- ◯ Não aplica teste de unidade
- ◯ Outro: _____

12. Quais são as ferramentas de teste de integração que você utiliza em seus projetos front-end? *

Marcar apenas uma oval.

- ◯ Cypress
- ◯ Selenium
- ◯ Puppeteer
- ◯ WebDriverIO
- ◯ Não aplica teste de integração
- ◯ Outro: _____

13. Você tem alguma sugestão de tecnologia ou ferramenta que não tenha sido mencionada acima? Se sim, qual?

_____

_____

_____

_____

_____

14. Quais são as métricas de desempenho que você costuma medir durante os testes no front-end? *

Marque todas que se aplicam.

- ☐ Tempo de carregamento da página
- ☐ Tempo de resposta do servidor
- ☐ Utilização de CPU
- ☐ Utilização de memória
- ☐ Outro: _____

15. Qual é o nível de satisfação que você ou sua equipe com a qualidade e funcionalidade do front-end? *

Marcar apenas uma oval.

	1	2	3	4	5	
insa	◯	◯	◯	◯	◯	Muito satisfeito

16. Qual é o nível de confiança que você ou sua equipe possuem no código front-end? *

Marcar apenas uma oval.

	1	2	3	4	5	
Baix	◯	◯	◯	◯	◯	Alto nível de confiança

17. Qual é o nível de confiança que você ou sua equipe possuem nos testes elaborados para o front-end? *

Marcar apenas uma oval.

	1	2	3	4	5	
Baix	◯	◯	◯	◯	◯	Alto nível de confiança

18. Sua empresa ou equipe enfrenta alguma dificuldade na implementação de testes de software no front-end? Se sim, selecione as opções que mais se enquadram no seu contexto.

Marque todas que se aplicam.

- [ ] Falta de conhecimento técnico em testes de software no front-end
- [ ] Falta de tempo para implementar testes de software no front-end
- [ ] Dificuldade em encontrar ferramentas adequadas para os testes de software no front-end
- [ ] Resistência à mudança por parte da empresa/equipe
- [ ] Outro: _____

Pular para a pergunta 29

Pesquisa Sobre o Teste de Software no Front-end

19. Se sua equipe ou empresa não utiliza testes de software no front-end, qual a principal razão para isso? *

Marcar apenas uma oval.

- ◯ Não há tempo suficiente para implementar testes de software
- ◯ Não há conhecimento suficiente sobre testes de software
- ◯ Não há orçamento suficiente para implementar testes de software
- ◯ Crença de que testes de software no front-end não são necessários
- ◯ Falta de suporte da gerência
- ◯ Outro: _____

20. Se sua equipe ou empresa não utiliza testes de software no front-end, você ou sua equipe já enfrentou problemas em relação à qualidade do código e/ou funcionalidade? *

Marcar apenas uma oval.

- ◯ Frequentemente
- ◯ Ocasionalmente
- ◯ Nunca encontrou problemas

21. Se sua equipe ou empresa não utiliza testes de software no front-end, você ou sua equipe já enfrentou problemas em relação à manutenção do código e/ou funcionalidade? *

Marcar apenas uma oval.

- ◯ Frequentemente
- ◯ Ocasionalmente
- ◯ Nunca encontrou problemas

22. Se sua equipe ou empresa não utiliza testes de software no front-end, você ou sua equipe já enfrentou problemas em relação à escalabilidade? *

Marcar apenas uma oval.

- ◯ Frequentemente
- ◯ Ocasionalmente
- ◯ Nunca encontrou problemas

23. Se sua equipe ou empresa não utiliza testes de software no front-end, como você ou sua equipe lida com a garantia da qualidade do front-end sem a utilização de testes de software? *

Marcar apenas uma oval.

- ◯ Utilizamos outros métodos, como revisão de código e testes manuais
- ◯ Confiamos no nível da habilidade dos desenvolvedores para produzir código de qualidade
- ◯ Não temos um método específico de garantia de qualidade do front-end
- ◯ Outro: _____

24. Se sua equipe ou empresa não utiliza testes de software no front-end, como você ou sua equipe identifica problemas no front-end antes do lançamento? *

Marcar apenas uma oval.

- ◯ Realizamos testes manuais
- ◯ Realizamos revisões de código
- ◯ Não temos um método específico de identificação de problemas
- ◯ Outro: _____

25. Qual é o nível de confiança que você ou sua equipe possuem no código front-end? *

Marcar apenas uma oval.

	1	2	3	4	5	
Baix	◯	◯	◯	◯	◯	Alto nível de confiança

26. Qual é o nível de satisfação da equipe ou empresa com a qualidade e funcionalidade do front-end? *

Marcar apenas uma oval.

	1	2	3	4	5	
insa	◯	◯	◯	◯	◯	Muito satisfeito

27. Por qual razão sua equipe ou empresa poderia decidir implementar testes de software no front-end? *

Marcar apenas uma oval.

- ◯ Melhorar a qualidade do código e/ou funcionalidade
- ◯ Facilitar a manutenção do código e/ou funcionalidade
- ◯ Melhorar a escalabilidade
- ◯ Outro: _____

28. Se sua equipe ou empresa decidir implementar testes de software, qual seria a principal preocupação com relação à implementação? *

Marcar apenas uma oval.

- ◯ Tempo de implementação
- ◯ Custo de implementação
- ◯ Dificuldade em encontrar profissionais qualificados
- ◯ Outro: _____

29. Há mais alguma coisa que você gostaria de compartilhar sobre a implementação de testes de software no front-end em sua empresa ou equipe? Qualquer comentário adicional será muito útil.

Este conteúdo não foi criado nem aprovado pelo Google.

Google Formulários

APÊNDICE B – FEEDBACK PARA O GUIA DE DESENVOLVIMENTO DE TESTES *FRONT-END* WEB

Feedback para o guia de desenvolvimento de testes Front-end Web

TERMO DE CONSENTIMENTO LIVRE E ESCLARECIDO

Foi elaborado um guia sobre **testes de software para front-end Web**, para ajudar desenvolvedores a iniciar os estudos, na prática de teste e melhorarem a qualidade de seus projetos. O guia aborda desde conceitos básicos até exemplos práticos, usando ferramentas como Jest e Cypress.

Ficarei feliz com seu feedback! Acesse o guia no link: https://rosluc.github.io/front_test_guide/

Esta pesquisa visa avaliar o guia sobre testes de software no front-end Web, ajudando a identificar a clareza, relevância e utilidade do conteúdo apresentado.

O participante poderá deixar o formulário a qualquer momento sem qualquer prejuízo. Também não serão fornecidas quaisquer compensações financeiras aos participantes.

Informações de contato dos pesquisadores:

Rosendo Lucas Santana da Costa: rosendo.costa@alunos.ufersa.edu.br

Alysson Filgueira Milanez: alysson.milanez@ufersa.edu.br

* Indica uma pergunta obrigatória

1. Li e concordo com o Termo de Consentimento Livre e Esclarecido (TCLE): *

   Marcar apenas uma oval.

   ◯ Sim *Pular para a pergunta 2*

   ◯ Não

2. Qual o seu nível de familiaridade com testes de software para front-end Web? *

Marcar apenas uma oval.

- ◯ Nenhum
- ◯ Iniciante
- ◯ Intermediário
- ◯ Avançado

3. Com que frequência você realiza testes unitários em projetos front-end? *

Marcar apenas uma oval.

- ◯ Nunca
- ◯ Raramente
- ◯ Às vezes
- ◯ Frequentemente
- ◯ Sempre

4. Qual a sua experiência com ferramentas de teste como Jest ou Cypress? *

Marcar apenas uma oval.

- ◯ Nenhuma
- ◯ Básica
- ◯ Intermediária
- ◯ Avançada

5. O conteúdo do guia foi claro e fácil de entender. *

Marcar apenas uma oval.

- ◯ Discordo totalmente
- ◯ Discordo
- ◯ Neutro
- ◯ Concordo
- ◯ Concordo totalmente

6. A organização do guia facilitou o aprendizado sobre testes de software. *

Marcar apenas uma oval.

- ◯ Discordo totalmente
- ◯ Discordo
- ◯ Neutro
- ◯ Concordo
- ◯ Concordo totalmente

7. A profundidade das explicações fornecidas no guia foi suficiente. *

Marcar apenas uma oval.

- ◯ Discordo totalmente
- ◯ Discordo
- ◯ Neutro
- ◯ Concordo
- ◯ Concordo totalmente

8. Os exemplos de código no guia foram úteis para entender os conceitos de testes. *

Marcar apenas uma oval.

- ◯ Discordo totalmente
- ◯ Discordo
- ◯ Neutro
- ◯ Concordo
- ◯ Concordo totalmente

9. Você estaria interessado em contribuir com melhorias para este guia através de pull requests (PR) em um projeto aberto? *

Marcar apenas uma oval.

- ◯ Não, não tenho interesse
- ◯ Talvez, se eu tiver tempo
- ◯ Sim, gostaria de contribuir

10. Com que probabilidade você recomendaria este guia para outros desenvolvedores? *

Marcar apenas uma oval.

	1	2	3	4	5	
Nad	◯	◯	◯	◯	◯	Muito provável

11. Em uma escala de 1 a 5, como você avalia o guia de maneira geral? *

Marcar apenas uma oval.

	1	2	3	4	5	
Muit	◯	◯	◯	◯	◯	Excelente

12. Você tem algum feedback ou sugestão para melhorar o guia?

Este conteúdo não foi criado nem aprovado pelo Google.

Google Formulários